

FACULTY OF ENGINEERING AND TECHNOLOGY

ACADEMIC SESSION APRIL 2025

SUBJECT CODE AND NAME: EPM2003 PROJECT MANAGEMENT AND ENGINEERING DESIGN

ASSESSMENT NAME: INITIAL REPORT

TITLE: IRRIGATION WATER TEMPERATURE AND PRESSURE MONITORING WITH AUTOMATED HOT WATER RELEASE CONTROL (ARGRICULTURE AUTOMATION)

DUE DATE: 26 June 2025

No.	STUDENT ID	STUDENT NAME
1	24013567	Oh Yu Qiao
2	24016453	Alif Meherin
3	23130420	Bibi Irrma Fatimah Takun
4	24020083	Nicholas Lee Yong Wei
5	24015224	Tay Li Yuen

IMPORTANT

The university requires students to adhere to submission deadlines for any form of assessment. Penalties are applied to unauthorized late submission of work. Coursework submitted after the deadline will be subjected to the prevailing academic regulations. Please check your respective programme handbook.

Academic Honesty Acknowledgement

“We **Oh Yu Qiao, Alif Meherin, Bibi Irma Fatimah Takun, Nicholas Lee Yong Wei & Tay Li Yuen (names)** verify that this submission contains entirely our own work. We have not consulted with any outside person or materials other than what was specified (an interviewee, for example) in this assessment or the syllabus requirements. Furthermore, we have not copied or inadvertently copied ideas, sentences, or paragraphs from another student, or the Internet. We are aware of the penalties (refer to the student handbook and undergraduate programme handbook) for any copying or collaboration on any assessment.”

Oh Yu Qiao, Alif Meherin, Bibi Irma Fatimah Takun, Nicholas Lee Yong Wei & Tay Li Yuen (25/6/2025)

Table of Content

Table of Content	2
1.0 Project Charter	6
1.1 Defining the problem/ need:	6
1.2 Objectives of project	7
1.3 Potential Solutions	7
1.4 Assumptions	8
1.5 Risks & Solutions	8
1.6 Success Criteria	9
1.7 Project Timeline	9
1.8 Project Budget	10
1.9 Roles and Responsibilities of Team	10
1.10 Stakeholders	11
1.11 Market Research	11
1.12 Project Vision	11
1.13 Sign-Off Section	12
2.0 Project Timeline	12
2.1 Timeline overview	12
2.2 Gantt Chart	12
2.3 Critical Path Calculations	13
3.0 Software and Components selections	15
3.1 Software: Coding/ Programming	15
3.2 Code explanation	20
3.3 ThingSpeak Cloud Storage	31
3.4 Blynk IOT	34
3.5 Components Selection for circuit	40
3.6 Schematic diagram/ circuit connection	41
3.7 Software selection for 3D modelling and printing	44
3.8 3D prototype design of casing	44
4.0 Purchasing/ Cost Optimization	48
4.1 Strategies & Impacts	48
4.2 Recommendations	48
4.3 Components Cost	49
5.0 Fee Summary	51
5.1 Introduction	51

5.2 Budget overview/Planned Budget	51
5.3 Actual Cost Incurred (as of 14th August 2025)	51
5.4 Allocation breakdown (Planned Budget vs Actual expenditures)	52
5.5 Variance Analysis.....	52
5.6 Expected Final price of product	54
5.7 Profit Margins/Final Selling Price	54
6.0 Reliability and durability considerations	55
6.1 3D modelling of casing	55
6.2 Circuit	55
6.3 Other components.....	56
7.0 Development challenges	57
7.1 3D Printing.....	57
7.2 DS18B20 Sensor Operation	58
7.3 Components troubleshooting.....	58
7.4 Power source.....	59
7.5 Leakage from pipes.....	60
8.0 Implementation.....	61
8.1 Power	61
8.2 Solar Panel	64
8.3 Battery-Life.....	65
8.4 Flow Rate and pressure test on valve	66
8.5 Blynk IOT Software	67
8.6 Possible Improvements.....	69
9.0 Closing Remarks	70

1.0 Project Charter

1.1 Defining the Problem/Need

1.2 Objectives of Project

1.3 Potential Solutions

1.4 Assumptions

1.5 Risks & Solutions

1.6 Success Criteria

1.7 Project Timeline

1.8 Project Budget

1.9 Roles and Responsibilities of Team

1.10 Stakeholders

1.11 Market Research

1.12 Project Vision

1.13 Sign-off Section

2.0 Project Timeline

2.1 Timeline Overview

2.2 Gantt Chart

2.3 Critical Path Calculation

3.0 Software and Components Selection

3.1 Software: Coding/Programming

3.2 Code Explanation

3.3 ThingSpeak

3.4 Blynk IOT

3.5 Components Selection for Circuit

3.6 Schematic diagram/Circuit Connection

3.7 Software Selection for 3D Modelling and Printing

3.8 3D Prototype Design of Casing

4.0 Purchasing/Cost Optimisation

4.1 Strategies & Impacts

4.2 Recommendations

4.3 Components Cost

5.0 Fee Summary

5.1 Introduction

5.2 Budget overview/Planned Budget

5.3 Actual Cost Incurred

5.4 Allocation Breakdown (Planned Budget vs Actual Expenditures)

5.5 Variance Analysis

5.6 Expected Final Price of Product

5.7 Profit Margins/Final Selling Price

6.0 Reliability and Durability Considerations

6.1 3D Modelling of Casing

6.2 Circuit

6.3 Other Components

7.0 Development Challenges

7.1 3D Printing

7.2 DS18B20 Sensor Operation

7.3 Components Troubleshooting

7.4 Power Source

7.5 Leakge from Pipes

8.0 Implementation

8.1 Power

8.2 Solar Panel

8.3 Battery Life

8.4 Flow Rate and Pressure Test on Valve

8.5 Blynk IOT Software

8.6 Possible Improvements

9.0 Closing Remarks

1.0 Project Charter

Project Title : IRRIGATION WATER TEMPERATURE AND PRESSURE MONITORING WITH AUTOMATED HOT WATER RELEASE CONTROL (ARGRICULTURE AUTOMATION)

Assigned Date : April 24, 2025

Project Manager : Oh Yu Qiao (Contact: +60111193213)

1.1 Defining the problem/ need:

Effective irrigation management is not only a task, but a necessity in a place like Malaysia, where the tropical climate brings high temperatures and intense sunlight all year long. During our research, we found one common yet overlooked issue in Malaysian agriculture: water stored in irrigation pipelines heats up under the intense Malaysian sun, often reaching over 50°C. When the hot water is suddenly released onto crops, it often causes many problems, including thermal shock, burns roots, and damages plant health, especially in fertigation systems used for vegetables. Generally, farmers do not have the equipment or the time to manually check the water's temperature or wait for the water to cool down before irrigation. Moreover, the number of small farmers tend to stick to implementing manual valve control, which are less consistent and labour-intensive.

We recognized the need for a cost-effective, automated system that:

- Detects harmful water temperature levels
- Stops hot water from reaching plants
- Uses clean energy
- Offers remote control and monitoring features

1.2 Objectives of project

1. For this project, our prime objective is to create an automated system that continuously measures the pressure and temperature of water during irrigation.
2. Alongside, using electronically controlled valves to stop the release of hot water during the hottest parts of the day.
3. And to use solar energy to run the system sustainably and eliminate the need for wired power.
4. Incorporating IoT capabilities to use cloud platforms for remote system monitoring and control.
5. Also, to create a weather-resistant, long-lasting, and reasonably priced casing that is appropriate for Malaysian outdoor settings.
6. Lastly to test the system on a small-scale crop field for evaluating its effectiveness in reducing plant stress.

1.3 Potential Solutions

We designed a smart irrigation controller that uses sensors to constantly monitor the water temperature and pressure. When the temperature goes beyond the safe limit (e.g., 35–40°C), the system either delays irrigation or flushes out the hot water before it reaches the plants.

Our solution mainly includes:

- ESP8266 NodeMCU microcontroller for processing and Wi-Fi communication
- Temperature Sensor – DS18B20 that detects ambient temperature which is supportive of the ESP8266
- Solenoid valve – Valve that opens and closes under a closed circuit.
- Relay Module which will act as a switch to control the solenoid valve
- Power Supply – Battery or solar power system (rechargeable battery with wiring connected to solar panels)
- PVC Piping – Connected to the main farming system of the soil.
- LCD – Display temp and system status (interface)
- 12V rechargeable battery charged via solar panel

Scope of project:

- Sensing water pressure and temperature
- Automated valve control with solenoid valve
- Making decisions using a microcontroller (ESP8266 or Arduino)
- Battery system that runs on solar power
- Real-time data display via LCD or cloud interface
- Crop field testing (e.g., leafy vegetables or chili)
- Waterproof enclosure (3D printed or commercially available IP-rated box)

1.4 Assumptions

A few assumptions have been made for optimised use of the system we developed:

- Sunlight is available on the farm for solar power.
- The field has hotspot or Wi-Fi connectivity.
- Farmers can use simple systems (check LCD, press reset).
- Irrigation pipes are either PVC or suitable for installing valves.
- Rain and humidity in Malaysia can be tolerated by component wiring and enclosures.

1.5 Risks & Solutions

Risk	Impact	Preventive Measures
Overheated water released	Plant damage	Temperature threshold logic cuts off hot water
Sensor failure	System won't respond	Waterproof sensors, regular calibration
Solar power failure	System shuts down	Battery backup and oversizing of panel
Leakage or short circuit	System failure	Use of IP-rated enclosure and sealants
Overheating of electronics	Microcontroller damage due to overheat	Add vent holes, place in shaded areas
Project cost too high	Budget issue	Prioritize local and student-discounted components

Table 1.5.1 Risk, impact & respective preventive measures

1.6 Success Criteria

We have listed the following success criteria to assess if our project has met its objectives. The system will be considered effective if:

1. **Water Overheating Prevention:** It accurately detects when the water temperature exceeds acceptable levels (35 to 40°C) and hot water supply to the crops is automatically shut off.
2. **Timely Valve Responsiveness:** The selected valve (solenoid or motorized) responds to the sensors by opening and closing within a short, predetermined time frame with no failures.
3. **Solar-Only Operation:** The system can operate fully on solar power, with a battery capable of sustaining operation during the day even on cloudy days.
4. **Accessible monitoring:** An LCD display or cloud-based mobile dashboard offers clear readability of temperature/pressure readings and system status for users or farmers.
5. **Improved Crop Protection:** In contrast to unregulated irrigation, crops irrigated by our system show less indication of root stress and keep growing healthier in field trials.
6. **Durable Outdoor Response:** Under actual farm conditions, each element (microcontroller, sensors, and power) shielded and continues to work within our weather-sealed housing.
7. **Affordability & Accessibility:** The system can be easily replicated or copied for small Malaysian farms and is affordable, ranging from RM 200 to RM 300.
8. **High Reliability, Low Maintenance:** The system must be installed and after that does not require any high maintenance and needs only periodic cleaning or recharging, hence ideal for long-term usage.

1.7 Project Timeline

This project is estimated to be completed within 12 weeks starting as from the 8th of May 2025. In weeks 1 and 2, the team focuses on the project analysis; that is, to gather information about the project, do thorough research and understand the problems faced by the farmers. In weeks 3 to 5, the necessary materials needed for this project are purchased. Right after that, week 3 to week 8 involves the writing of the code, testing the system and making sure the code works correctly. The 3D modelling is conducted during weeks 6 to 8 and the circuitry simulation, connection and soldering span over a period of 6 weeks that is, weeks 3 to 9. Afterwards, the

products are tested individually and as a whole system during weeks 11 to 12 and the analysis of results is performed to check for the outcomes and data from the project. The financial and initial reports are drafted during the weeks 4 to 5 and the whole of the second quarter duration of the whole project's duration. Finally, the team prepares for the final submission and the presentation in weeks 11 to 12.

1.8 Project Budget

The total budget for this project is RM 150. This amount covers all the necessary expenses of the materials to be used to complete the project, excluding the solar panel, the ESP8266 microcontroller, the relay, the 3D printing and the LCD with the I2C module.

1.9 Roles and Responsibilities of Team

To ensure equal work distribution, the below roles are discussed and assigned amongst our group members:

Roles	Responsibilities
Project Manager and Software Engineer: 1. Oh Yu Qiao	• Person in charge for the execution of the project. • Provides proper coordination of tasks among group members. • Responsible for the coding part of the project.
Product Designers: 1. Tay Li Yuen 2. Nicholas Lee Yong Wei	• Responsible for the designs of the 3D modelling of the product. • Provide backup for hardware connections.
Hardware Technicians: 1. Alif Meherin 2. Bibi Irrma Fatimah Takun	• Responsible for simulations. • Responsible for the circuitry connections and soldering.

Table 1.9.1 Roles and responsibilities of all group members

1.10 Stakeholders

- Team members of the project : Oh Yu Qiao, Tay Li Yuen, Nicholas Lee Yong Wei, Alif Meherin and Bibi Irrma Fatimah Takun.
- Project Supervisor : Ker Pin Jern
- Project Sponsor : Sunway University
- Farmers

1.11 Market Research

In Malaysia, farmers have started to include the use of technology to grow crops in the agricultural sector to reduce the dependency of foreign workers and the difficulty in finding enough skilled workers to work in farms. A common farming method is fertigation, which involves the mixing of fertilisers with water which are to be sent to plants through pipes and the plants directly absorb the nutrients as they are already dissolved in water. But due to issues such as heat accumulation in the exposed pipes as they are left under the sun, heat stress can be caused to plants as the hot water is used, ultimately reducing the crop yield. Because of this problem, many farmers are looking for better systems of checking the temperature and fixing this problem automatically. Hence, the market is ready for advanced irrigation systems that can be used widely to solve this challenge.

1.12 Project Vision

The project vision is to build a smart farming system that will not only check for the temperature of the water in the pipes that will reach the plants but will also ensure the safety and a good yield for the plants. For instance, if the water is too hot, the system will release it before it even reaches the plants so that the roots are protected and the plants are kept healthy. This system will also send the data regarding the temperature to an online storage and hence, it can be managed and controlled using merely a device such as a phone or a timer. The design will also be made waterproof so that it can be kept outdoors and will also use batteries so that it can work even without electricity. At last, by using this project in farms, a better yield of crops will be grown without the need of many farmers.

1.13 Sign-Off Section

Oh Yu Qiao (Project Manager and Software Engineer): _____ **Joshua** _____ Date: 17/8/2025

Tay Li Yuen (Product Designer) : _____ **Li Yuen** _____ Date: 17/8/2025

Nicholas Lee Yong Wei (Product Designer): _____ **Nicholas** _____ Date: 17/8/2025

Alif Meherin (Hardware Technician): _____ **Alif** _____ Date: 17/8/2025

Bibi Irrma Fatimah Takun (Hardware Technician): _____ **Irrma** _____ Date: 17/8/2025

2.0 Project Timeline

2.1 Timeline overview

To tackle the task more effectively, the main task is broken down into smaller components. The ideal project duration is calculated to be 12 weeks, through the implementation of formal project management methods, namely Critical Path Method (CPM) and Gantt Charts.

Project Issuance Date: 8th May 2025

2.2 Gantt Chart

A Gantt Chart allows visual representation of tasks in the form of a timeline. Through the Gantt Chart, we are able to improve the productivity of the team and internal coordination between group members is enhanced. Not only that, but deadlines can also be better visualised, hence preventing unnecessary delays and procrastination. Team morale can also be boosted with the clear visualisation of tasks.

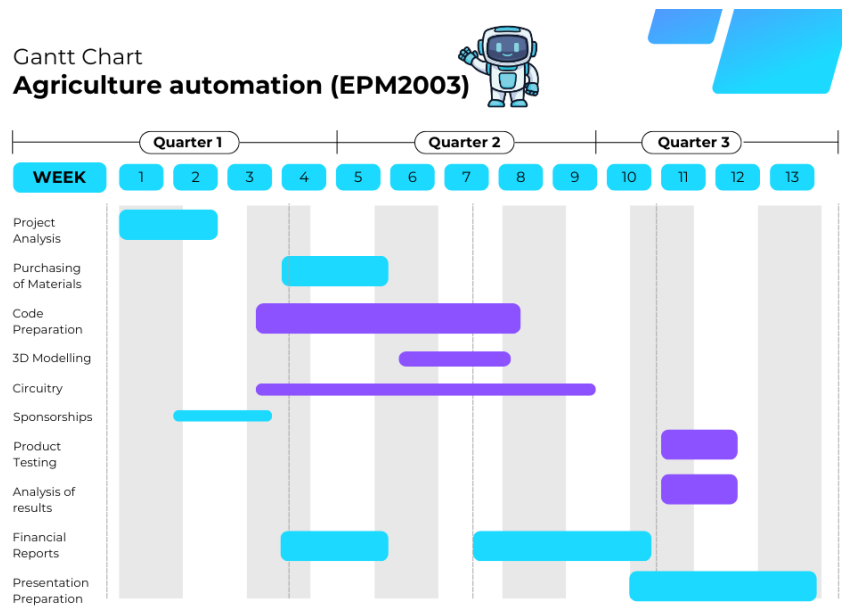


Figure 2.2.1 Gantt Chart

2.3 Critical Path Calculations

Critical Path Method allows us to identify the longest time required for a project from start to finish, to identify potential challenges. The progress of each task can be tracked, and critical tasks or phases will be given higher priority to avoid delays on the entire project. CPM in coordination with the Gantt Chart shall provide sufficient information on the progress of the entire project, to ensure the project can be finished on time. Project duration is calculated to be 13 weeks.

Activity ID	Phase/Stage
A	Project Analysis
B	Purchasing of Materials
C	Code Preparation
D	3D Modelling
E	Circuitry
F	Sponsorships
G	Product Testing
H	Analysis of Results
I	Financial Reports
J	Presentation Preparation

Table 2.2.2 Activity ID Representation for CPM

ID	Representation	Predecessors	Duration (Weeks)	ES	EF	LS	LF	Slack	Critical
A	Project Analysis	-	2	0	2	0	2	0	YES

B	Purchasing of Materials	A	2	2	4	11	13	9	NO
C	Code Preparation	A	5	2	7	3.5	8.5	1.5	NO
D	3D Modelling	A	2	2	4	6.5	8.5	4.5	NO
E	Circuitry	A	6.5	2	8.5	2	8.5	0	YES
F	Sponsorships	-	2	0	2	5	7	5	NO
G	Product Testing	C, D, E	1.5	8.5	10	8.5	10	0	YES
H	Analysis of Results	D, E	1.5	8.5	10	8.5	10	0	YES
I	Financial Reports	F	6	2	8	7	13	5	NO
J	Presentation Preparation	G, H	3	10	13	10	13	0	YES

Table 2.2.3 Tabulation of calculating CPM

ES = Early Start
EF = Early Finish
LS = Late Start
LF = Late Finish

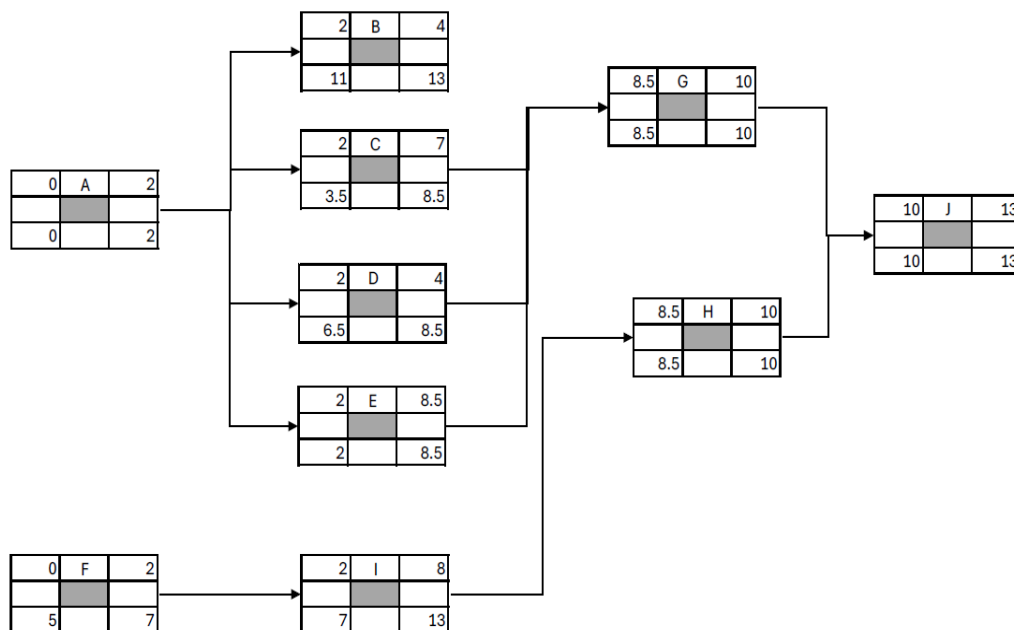


Figure 2.2.4 Flow of Activities in CPM

The above diagram shows the flow of activities using a spreadsheet and flowchart combination for better visualisation. The top left of each box shows early start (ES), while the top right shows early finish (EF). As for the bottom left of each box, late start (LS) is shown, while the bottom right shows late finish (LF). Relationships between activities have also been clearly indicated in the graph using Arrow Diagramming Method (ADM).

3.0 Software and Components selections

3.1 Software: Coding/ Programming

In terms of software for coding and the programming part for the operation of the Node MCU ESP8266 microcontroller, we have decided to go with Visual Studio code with the extension of platformio. We chose VS code instead of straight coding with the Arduino IDE software because the environment for code development in VS code is much more powerful and flexible compared with the Arduino IDE, for example in VS code we can open multiple files at once in a single tab, while the Arduino IDE does not allow that, which would prove to be important when multitasking:

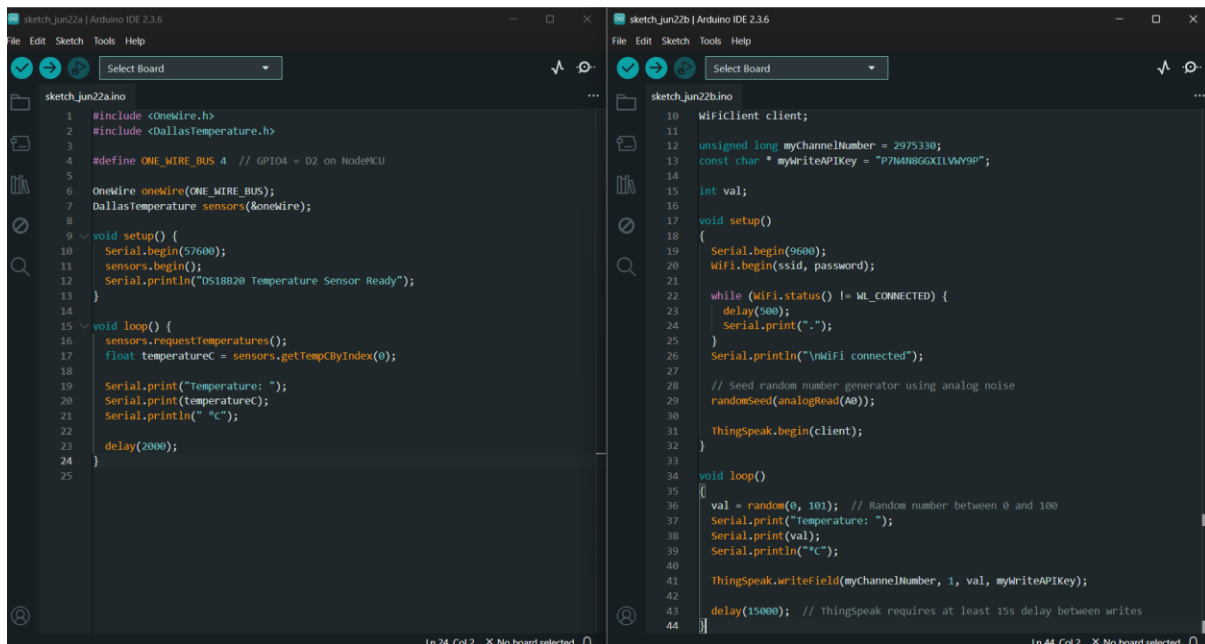


Figure 3.1.1: Arduino IDE does not allow opening multiple files in the same tab

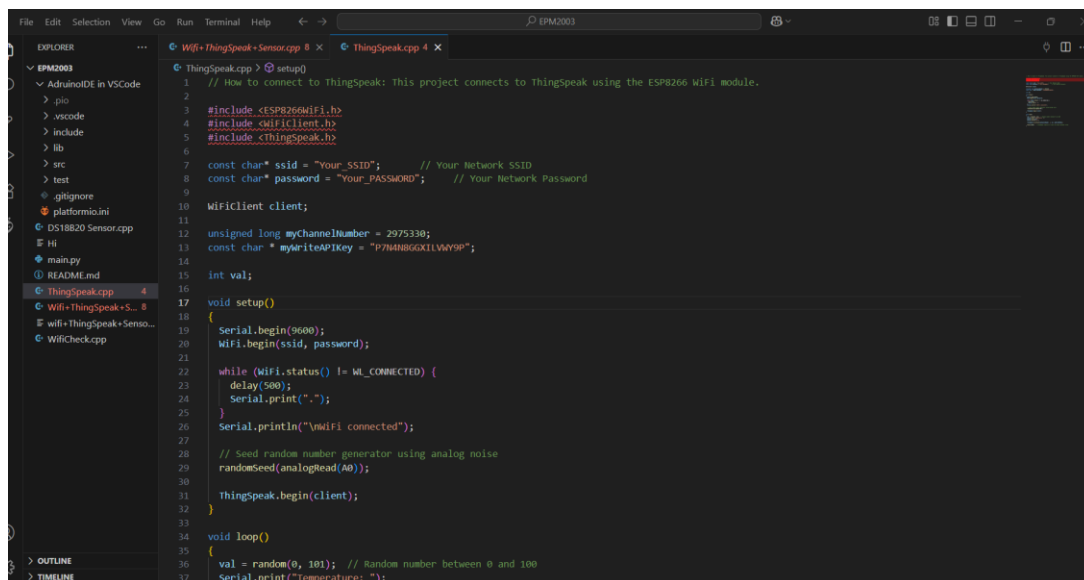


Figure 3.1.2: VS code allows for opening multiple files in one tab

Apart from that, VS code offers features such as intelligent code completion (IntelliSense) with better debugging tools and custom extensions which enhance productivity and make code management easier especially for project such as this:

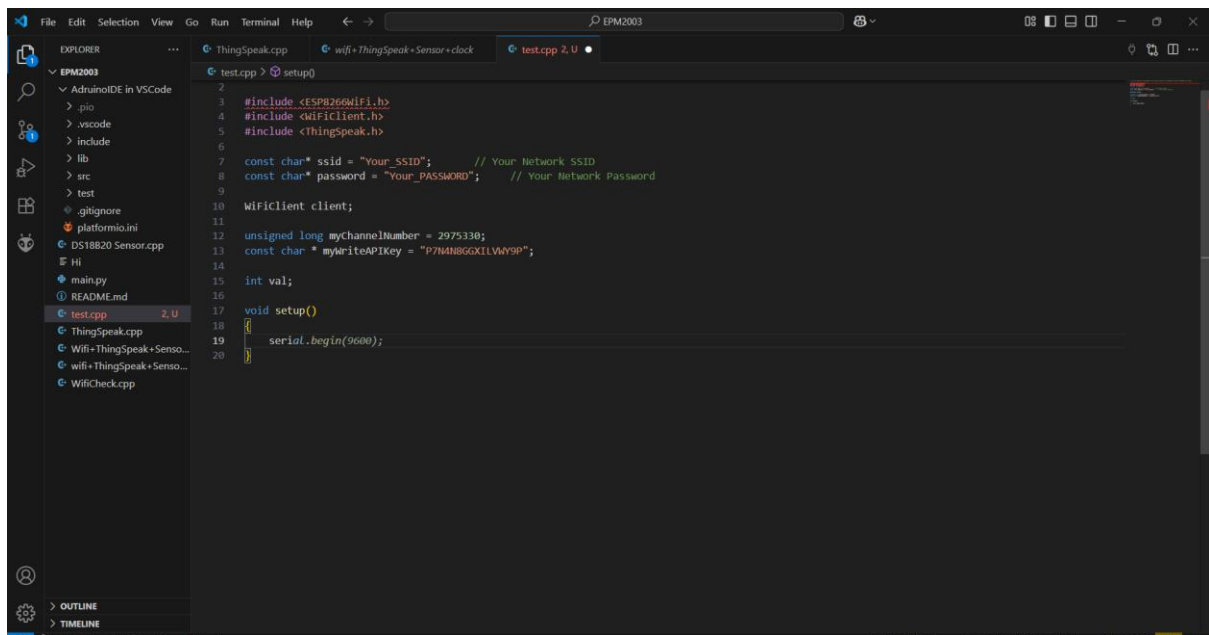


Figure 3.1.2: VS code allows for easier and better coding experience

Meanwhile, if we were to be using only the Arduino IDE, code debugging would not have more dynamic compared to VS code.

But the most important factor of why we chose VS code is the easy integration of Git and GitHub which was pivotal when it came to the management part of the coding for this project. While the Arduino IDE is suitable for smaller projects and basic coding for microcontrollers, it lacks modularity and tools for modern version control which are critical to our efficient workflow. Using Git within VS Code allowed us to efficiently track changes via the different branch off changes features which maintain a clean and recoverable development history if something goes wrong all directly from the VS Code software:

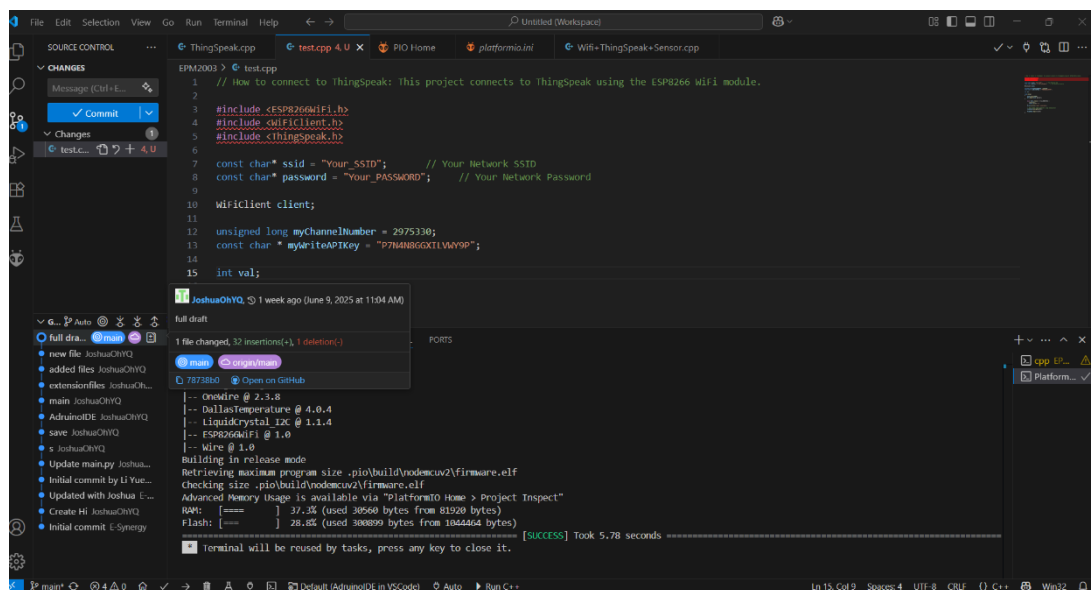


Figure 3.1.3: VS code allows for branching and history tracking via cloud (Github)

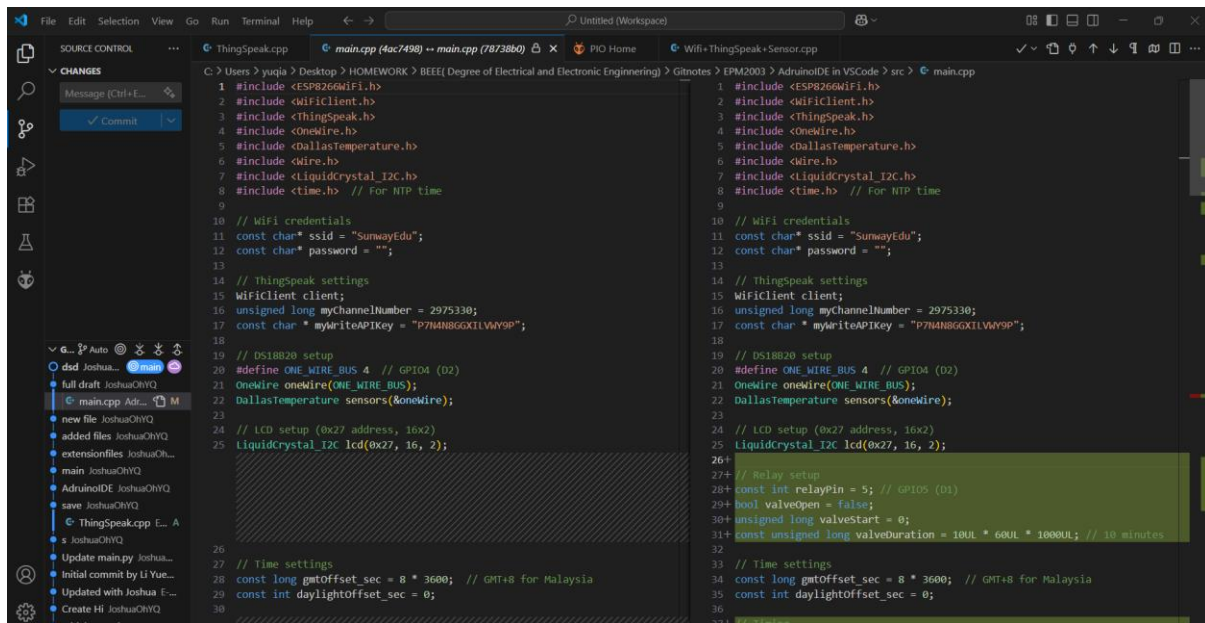


Figure 3.1.4: VS code with GitHub allows for tracking of different changes made by different users at different commits or branches

This feature became important as our code evolved into a more complete cohesive and complex system with multiple cpp interdependent files and many modules that handled different functionalities such as sensor integration, relay logic control, Blynk IOT and etc. How we made the code is, we did it part by using creating different files, for example the DS18B20 temperature probe sensor was tested first and coded in a file, then we moved on to the relay logic which was coded and tailored to time settings in a separate file. Using VS code made this so much easier as, we can open multiple files at once and get tracking history of our progress via GitHub.

Furthermore, one of the most important functions in GitHub is, it can be considered as a centralized, cloud-based repository where all the group members could collaborate in real time with hassle. This allows everyone in the repository to review each other commits, raise issues and pull requests for improvements. This greatly enhanced our teamwork and the overall coordination, which is one of the key points in the concept of CDIO (conceive, design, implement and operate), that is working in a team in a holistic manner, making sure that all the code were systematically integrated. Besides, GitHub is more than a cloud repository website, it also allows us to assign tasks and set milestones, which is crucial to following the schedule set in the Gantt chart:

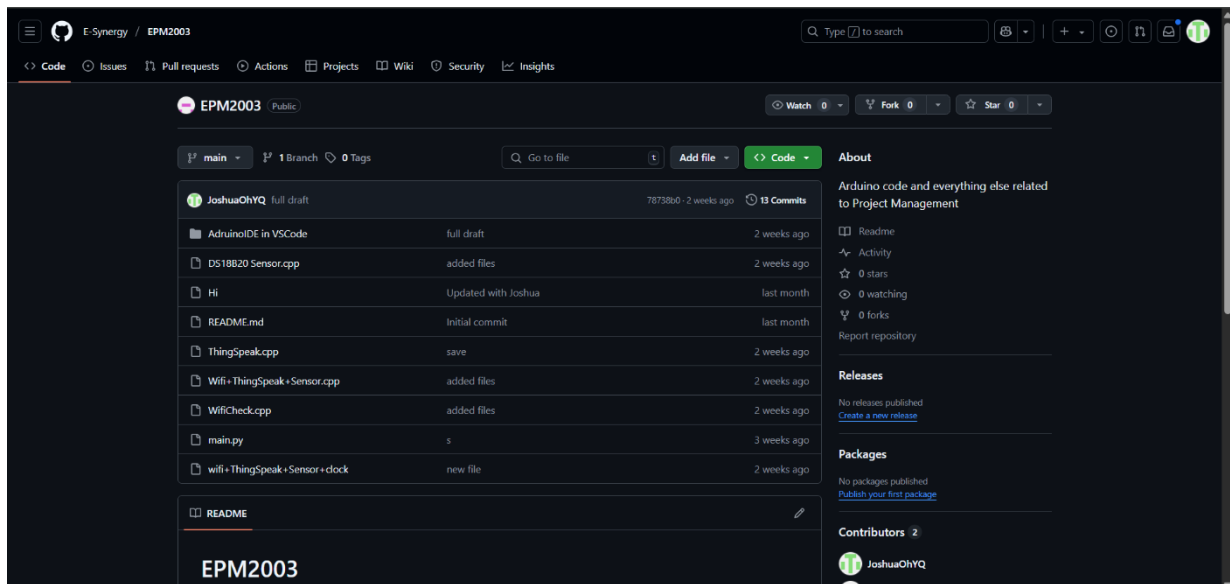


Figure 3.1.5: GitHub repository page for our project

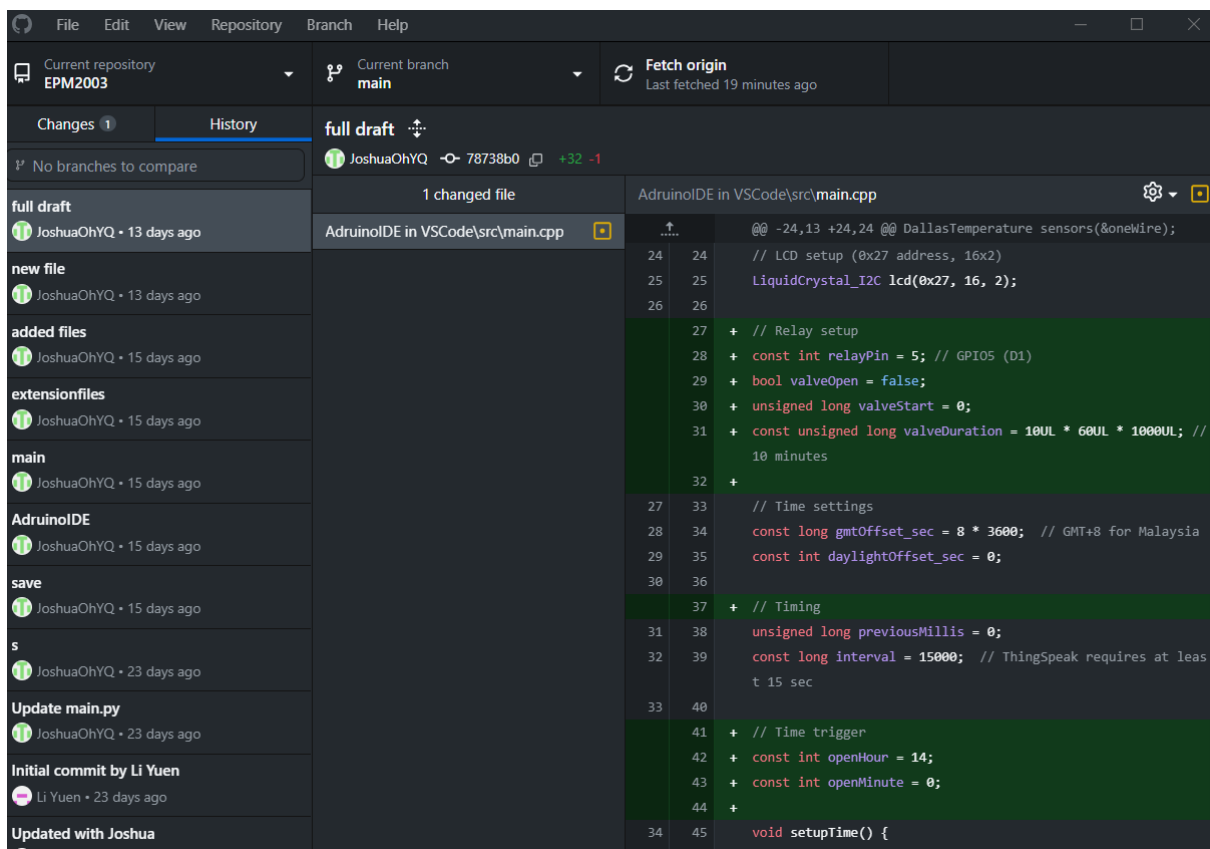


Figure 3.1.6: GitHub Desktop which can track commits and many other different actions made by our team members

Another key benefit of VS Code with GitHub integration is the ability to revert to previous working states which is very important especially during debugging stages of the code and experimental features during the Implementation and Operation phases of CDIO. This allowed us to safely debug and test the code for every individual component in isolation without risking the integrity of the whole system and if one thing messes up, we can just revert to the previous stage easily just by pressing a button.

On the hand, to run the code on our Node MCU ESP8266 microcontroller, we used Platformio which is an advanced development ecosystem built on VS Code. It offers plenty of build in system, board and framework abstraction, automatic library dependency management and multiple embedded platforms support including the ESP 8266. Compare with the Arduino IDE which relies on manual configuration, Platformio handles all aspects of coding highly organized making it easy to replicate, share and maintain the coding project:

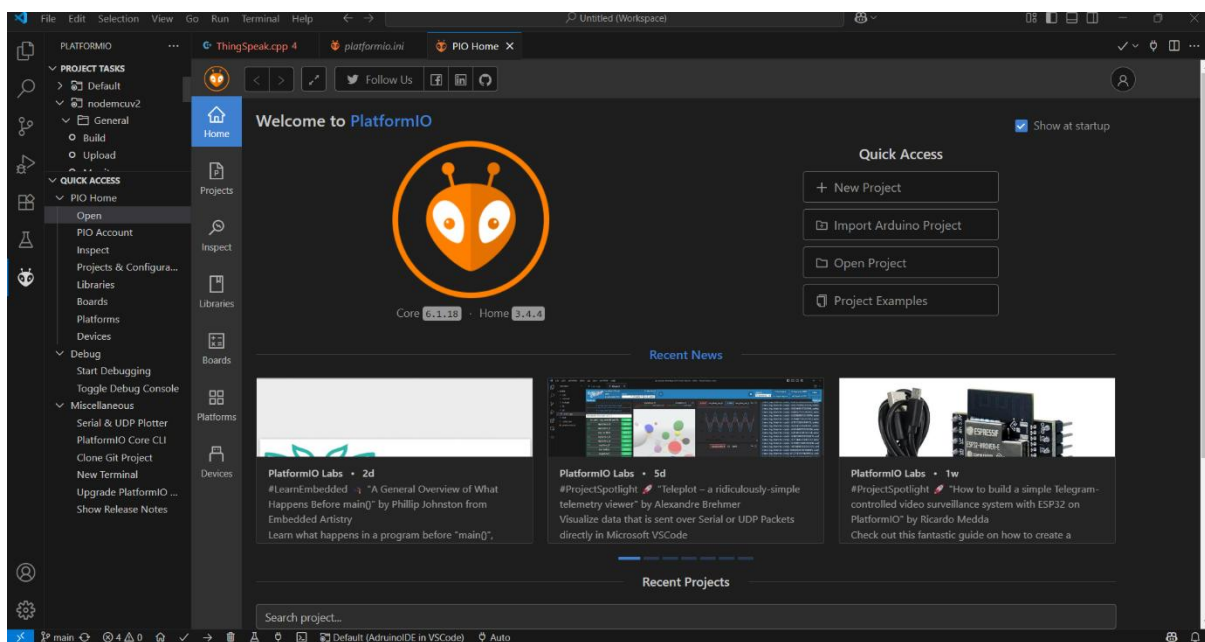


Figure 3.1.7: Integration of Platformio with VS Code

3.2 Code explanation

The full code for the agriculture automation system:

```
// ===== Blynk Template Info =====
#define BLYNK_TEMPLATE_ID "TMPL6eoSfctzR"
#define BLYNK_TEMPLATE_NAME "IOT Agriculture"
#define BLYNK_AUTH_TOKEN "s0uaYLq3KStgyuW9zWfVMcF2iMigEtML"

// ===== Libraries =====
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <time.h>

// ===== WiFi credentials =====
char ssid[] = "SunwayEdu";
char pass[] = "";

// ===== Pin assignments =====
#define ONE_WIRE_BUS D2      // DS18B20 data pin (GPIO4)
#define RELAY_PIN D5        // Relay control pin (GPIO14)

// ===== LCD I2C settings =====
LiquidCrystal_I2C lcd(0x27, 16, 2); // 16x2 LCD, I2C address 0x27

OneWire oneWire(ONE_WIRE_BUS);
DallasTemperature sensors(&oneWire);
BlynkTimer timer;

// ===== States =====
bool autoMode = false;
bool relayState = false;

// ===== Timezone for Malaysia =====
const long gmtOffset_sec = 8 * 3600;
const int daylightOffset_sec = 0;

// ===== Blynk Controls =====

// Manual relay control (V1)
BLYNK_WRITE(V1) {
  relayState = param.asInt();
  digitalWrite(RELAY_PIN, relayState ? HIGH : LOW);
  Serial.println(relayState ? "Relay: MANUAL ON" : "Relay: MANUAL OFF");
}
```

```

// Auto mode toggle (V2)
BLYNK_WRITE(V2) {
    autoMode = param.asInt();
    Serial.println(autoMode ? "AUTO MODE: ON" : "AUTO MODE: OFF");
}

// ===== LCD Display Function =====

void updateDisplay(float tempC, const char* relayStatus, const char* timeStr)
{
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Time:");
    lcd.print(timeStr);

    lcd.setCursor(15, 0); // Show 'A' or 'M' at top-right
    lcd.print(autoMode ? "A" : "M");

    lcd.setCursor(0, 1);
    lcd.print("T:");
    lcd.print(tempC);
    lcd.print((char)223); // Degree symbol
    lcd.print("C ");
    lcd.print(relayStatus);
}

// ===== Temperature Check Function =====

void checkTemperature() {
    sensors.requestTemperatures();
    float tempC = sensors.getTempCByIndex(0);

    // Handle sensor error
    if (tempC == DEVICE_DISCONNECTED_C) {
        Serial.println("Error: DS18B20 not detected!");

        // Show error on LCD
        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Sensor Error!");
        lcd.setCursor(0, 1);
        lcd.print("Check DS18B20");

        // Also send error flag to Blynk (optional)
        Blynk.virtualWrite(V0, -127); // convention: -127 = error

        return;
    }
}

```

```

}

Blynk.virtualWrite(V0, tempC); // Send temp to Blynk gauge/chart

// Get current time
time_t now = time(nullptr);
struct tm* timeinfo = localtime(&now);
char timeStr[9]; // HH:MM:SS
strftime(timeStr, sizeof(timeStr), "%H:%M:%S", timeinfo);

// Auto mode logic
if (autoMode && tempC > 30.0 && !relayState) {
    digitalWrite(RELAY_PIN, HIGH);
    relayState = true;
    Serial.println("AUTO: Relay ON due to temp > 30°C");
} else if (autoMode && tempC <= 30.0 && relayState) {
    digitalWrite(RELAY_PIN, LOW);
    relayState = false;
    Serial.println("AUTO: Relay OFF, temp normal");
}

const char* relayStatus = relayState ? "ON " : "OFF";
updateDisplay(tempC, relayStatus, timeStr);
}

// ===== NTP Time Sync =====

void setupTime() {
    configTime(gmtOffset_sec, daylightOffset_sec, "pool.ntp.org",
"time.nist.gov");
    Serial.print("Waiting for NTP");
    while (time(nullptr) < 100000) {
        Serial.print(".");
        delay(500);
    }
    Serial.println("\nTime sync complete!");
}

// ===== Setup =====

void setup() {
    Serial.begin(9600);

    pinMode(RELAY_PIN, OUTPUT);
    digitalWrite(RELAY_PIN, LOW); // Start with relay OFF

    sensors.begin();

```

```
Wire.begin(D3, D4);           // Use D3 (GPIO0) for SDA, D4 (GPIO2) for SCL
lcd.init();
lcd.backlight();
lcd.setCursor(0, 0);
lcd.print("Connecting...");

Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

setupTime();

// Sync saved states from Blynk
Blynk.syncVirtual(V1);
Blynk.syncVirtual(V2);

timer.setInterval(5000L, checkTemperature); // every 5 seconds
}

// ===== Loop =====

void loop() {
  Blynk.run();
  timer.run();
}
```

Code explanation by parts:

```
#define BLYNK_TEMPLATE_ID "TMPL6eoSfctzR"
#define BLYNK_TEMPLATE_NAME "IOT Agriculture"
#define BLYNK_AUTH_TOKEN "s0uaYLq3KStgyuW9zWfVMcF2iMigEtML"
```

This part establishes the project on Blynk IoT Platform. The template ID and template name points to a pre-designed layout template on the Blynk mobile app. With the unique authorisation token, a secure, authenticated connection can be established between the microcontroller and the Blynk cloud server.

```
#include <ESP8266WiFi.h>
#include <BlynkSimpleEsp8266.h>
#include <OneWire.h>
#include <DallasTemperature.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <time.h>
```

This part includes all the software libraries required for our code. Below is a brief description of each's functionalities.

Library(s)	Description
ESP8266WiFi.h	Provides the functions to connect the ESP8266 module to a Wi-Fi network.
BlynkSimpleEsp8266.h	Simplifies the process of sending and receiving data from the Blynk cloud using the ESP8266.
OneWire.h DallasTemperature.h	Communicate with and read temperature data from the DS18B20 temperature sensor.
Wire.h	Enables I2C communication for LCD Display.
LiquidCrystal_I2C.h	Provides functions to control the I ² C LCD display easily (showing text, clearing screen, etc.)
Time.h	Used to fetch, manage, and format the current time from Network Time Protocol (NTP) servers.

```
char ssid[] = "SunwayEdu";
char pass[] = "";
```

This part stores the credentials for the local wireless network that the device will connect to.

```
#define ONE_WIRE_BUS D2 // DS18B20 data pin (GPIO4)
#define RELAY_PIN D5 // Relay control pin (GPIO14)
```

This part of the code maps the physical connections on the ESP8266 board to named constants in the code, where DS18B20 sensor's data is sent into the D2 digital pin of the ESP8266, whereas the relay is controlled with the D5 digital pin of the microcontroller to complete the circuit for the solenoid valve.

```
LiquidCrystal_I2C lcd(0x27, 16, 2);  
  
OneWire oneWire(ONE_WIRE_BUS);  
DallasTemperature sensors(&oneWire);  
BlynkTimer timer;
```

The LCD is initialised with an address of 0x27, and a size of 16 columns x 2 rows. A OneWire communication object is also created on the pin defined earlier in the code (D2). DallasTemperature sensor object is also created in conjunction with the OneWire object to obtain the temperature readings. A timer is used to run periodic tasks without blocking the main loop.

```
bool autoMode = false;  
bool relayState = false;
```

This part introduces two state variables to track the operational state of the system. autoMode is set to false by default to see if whether the system is in automatic mode (where it controls the relay based on temperature) or manual mode (where the user controls it via the app), whereas relayState, set to false by default as well, tracks whether the relay is operating.

```
const long gmtOffset_sec = 8 * 3600;  
const int daylightOffset_sec = 0;
```

This part configures the local time settings for the device. The product is set to operate in Greenwich Mean Time (GMT) +8, which is the time zone for Malaysia.

```
BLYNK_WRITE(V1) {  
    relayState = param.asInt();  
    digitalWrite(RELAY_PIN, relayState ? HIGH : LOW);  
    Serial.println(relayState ? "Relay: MANUAL ON" : "Relay: MANUAL OFF");  
}  
  
BLYNK_WRITE(V2) {  
    autoMode = param.asInt();  
    Serial.println(autoMode ? "AUTO MODE: ON" : "AUTO MODE: OFF");  
}
```

These functions define how the device responds to commands sent from the Blynk mobile app. Two virtual pins on Blynk have been assigned. V1 is assigned to relayState flag, while V2 is assigned to autoMode flag. param.asInt() converts the command (i.e. user input) from the app into an integer value of 1 or 0, which corresponds to ON or OFF.

```
void updateDisplay(float tempC, const char* relayStatus, const char* timeStr)
{
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Time:");
    lcd.print(timeStr);

    lcd.setCursor(15, 0);
    lcd.print(autoMode ? "A" : "M");

    lcd.setCursor(0, 1);
    lcd.print("T:");
    lcd.print(tempC);
    lcd.print((char)223);
    lcd.print("C ");
    lcd.print(relayStatus);
}
```

This part of the code consolidates all information onto the LCD in a formatted way. At column 0, row 0, the live time would be printed. At column 15, row 0, an A or an M would be printed based on the operating mode of the system. While at column 0, row 1, the temperature at the specific time would be printed out with the degree symbol.

```
void checkTemperature() {
    sensors.requestTemperatures();
    float tempC = sensors.getTempCByIndex(0);

    if (tempC == DEVICE_DISCONNECTED_C) {
        Serial.println("Error: DS18B20 not detected!");

        lcd.clear();
        lcd.setCursor(0, 0);
        lcd.print("Sensor Error!");
        lcd.setCursor(0, 1);
        lcd.print("Check DS18B20");

        Blynk.virtualWrite(V0, -127);

        return;
    }
}
```

```

}

Blynk.virtualWrite(V0, tempC);

time_t now = time(nullptr);
struct tm* timeinfo = localtime(&now);
char timeStr[9];
strftime(timeStr, sizeof(timeStr), "%H:%M:%S", timeinfo);

if (autoMode && tempC > 30.0 && !relayState) {
    digitalWrite(RELAY_PIN, HIGH);
    relayState = true;
    Serial.println("AUTO: Relay ON due to temp > 30°C");
} else if (autoMode && tempC <= 30.0 && relayState) {
    digitalWrite(RELAY_PIN, LOW);
    relayState = false;
    Serial.println("AUTO: Relay OFF, temp normal");
}

const char* relayStatus = relayState ? "ON " : "OFF";
updateDisplay(tempC, relayStatus, timeStr);
}

```

This is the core function that runs periodically to read the sensor, make decisions, and update outputs. Firstly, it fetches the temperature value from the DS18B20, then it checks if the sensor is disconnected, if yes, an error is displayed on the LCD, and the serial monitor to indicate troubleshooting required. After everything regarding the sensor measurements are in place, the temperature value is sent to the Blynk cloud, to update the graph on the app.

If the auto mode is on, two scenarios are to be considered to control the valve.

Scenario 1: Temperature $\geq 30^{\circ}\text{C}$ & relay is off. Based off this condition, relay is turned on, valve goes on.

Scenario 2: Temperature $< 30^{\circ}\text{C}$ & relay is on. Based off this condition, relay is turned off, valve goes off.

All the information regarding the temperature, relay status and the time is updated accordingly on the LCD.

```

void setupTime() {
    configTime(gmtOffset_sec,    daylightOffset_sec,    "pool.ntp.org",
"time.nist.gov");
    Serial.print("Waiting for NTP");
    while (time(nullptr) < 100000) {
        Serial.print(".");
        delay(500);
    }
    Serial.println("\nTime sync complete!");
}

```

This part of the code synchronises the device’s internal clock with internet time servers using the Network Time Protocol – NTP. “pool.ntp.org” and “time.nist.gov” are hostnames of NTP servers that are highly reliable. A loop is implemented for a time sync for verification by comparing the pulled data with the standard time storage January 1, 1970, Unix Epoch figure.

```

void setup() {
    Serial.begin(9600);

    pinMode(RELAY_PIN, OUTPUT);
    digitalWrite(RELAY_PIN, LOW);

    sensors.begin();

    Wire.begin(D3, D4);
    lcd.init();
    lcd.backlight();
    lcd.setCursor(0, 0);
    lcd.print("Connecting...");

    Blynk.begin(BLYNK_AUTH_TOKEN, ssid, pass);

    setupTime();

    Blynk.syncVirtual(V1);
    Blynk.syncVirtual(V2);

    timer.setInterval(5000L, checkTemperature);
}

```

This function runs once at startup to initialise the entire system. Firstly, the serial communication is started, the relay pin is configured as an output and made sure it was LOW/OFF. The temperature sensor library, I2C Communication pins of D3, D4 are initialised as well. Then, the system connects to the Blynk server through Internet connectivity, and time is synchronised from the Internet. The latest states of the virtual toggles of V1 and V2 are also

fetches from Blynk App, and the timer is set to check the temperature every 5000 milliseconds (5 seconds).

```
void loop() {  
  Blynk.run();  
  timer.run();  
}
```

This function runs repeatedly after setup. It is the main runtime loop. The connection with Blynk cloud is constantly maintained and the check temperature function is constantly being called with reference to the set timer interval in the setup function, which was set to be 5000 milliseconds.

Code explanation in summary:

Basically, the idea behind the code is it runs on an ESP8266 microcontroller and performs temperature monitoring and either auto or manual valve control on Blynk for the water used in agriculture with real-time data logging to our cloud channel in Blynk:

First, the ESP8266 connects to the predefined WIFI network in this case SunwayEdu. It then reads the data from a DS18B20 digital temperature sensor every 5 to 8 seconds, which is then displayed on the I2C LCD screen (**in our case we removed it but if there was one it should still work**) and uploaded to the Blynk cloud servers using our own API. Furthermore, it also fetches the current time using NTP servers synchronized to GMT +8 which is the official Malaysian time and displays the current time on the LCD (**removed but should work if the LCD was plugged in again**). The LCD (**removed but should work if the LCD was plugged in again**) also displays errors if the system runs into any error, for example sensor error if DS18B20 is not detected or WIFI is not connected.

Since we decided to remove the I2C liquid crystal LCD display, the feature on these aspects using the LCD screen should would not be available to this product, however since the code was designed to be modular, the logic will not change much just that the features shown by the LCD screen will not be available, everything else will still be the same, but if the user plugged in a I2C liquid crystal LCD display into the system, it should function fully just like before.

The flow of the program goes like this:

- 1) Setup:
 - (a) Initialize hardware (WIFI, LCD, Sensor and time)
 - (b) Connect to Blynk
- 2) Loops:
 - (a) Every 5 seconds (due to latency delay could be around 5 to 8 seconds):
 - (i) Reads and displays temperature from sensor
 - (ii) Send temperature data to Blynk
 - (iii) Fetch data on current time and display it on LCD screen (removed, but still works if it was plugged in)
 - (b) If auto-mode is OFF and user turn ON valve manually via Blynk IOT widgets:
Turn ON the relay to open the valve or remain as it is if the relay is already ON
 - (c) If auto-mode is OFF and user turn OFF valve manually via Blynk IOT widgets:
Turn OFF the relay to close the valve or remain as it is if the relay is already OFF
 - (d) If auto-mode is ON and the temperature is below the threshold of 30 °C (this is during testing, in actual will be 38°C):
Turns OFF the relay to close the valve or remain as it is if the relay is already OFF
 - (e) If auto-mode is ON and the temperature is above or equals to the threshold of 30 °C (this is during testing, in actual will be 38°C):
Turns ON the relay to open the valve or remain as it is if the relay is already ON

3.3 ThingSpeak Cloud Storage

In terms of cloud storage and Internet of Things (IOT), we have decided to go with ThingSpeak, as advised by our lecturer, of which it provides several built-in tools not just for collecting and storing data, but it also enables extension with MATLAB for further analysis.

However in our case, we do not require MATLAB to analyse our data. However, one thing to note is we can use this compatibility with MATLAB to find out what is the trend of temperature throughout the day and throughout each season and made changes and modification for the time setting to turn ON/OFF the relay connected to the valve in this temperature monitoring system.

In terms of storing data and visualisation of data, ThingSpeak automatically generates live charts and graphs from sensor data which gives user of this agriculture automation system the full experience of the 21st century IOT features which has become a trend in our everyday lives. The live charts also help monitor trends and debugging issues when needed. Furthermore, ThingSpeak easy integration with our Node MCU ESP8266 which only require just a few lines of code to send data to the ThingSpeak channel. ThingSpeak also allow for event triggers and alerts that allow us to control devices or send alerts to email, for example when the temperature of the water is too high. It also allows for remote and portable usage through the Thingview app, which can be downloaded on Android and IOS, which basically works just as same as the desktop PC version. But one of the most important factors on why we chose ThingSpeak is its user friendliness as it removes the need to manage a backend or database like most software nowadays which save time, cost and maintenance efforts. It is also much simpler to understand for our customers which are the farmers, which may be not as skilled in the technology department, so this user friendliness not only gets them the latest IOT functions, but it is also very easy for them to use and implement into their system without having to know very technical stuff. In our opinion, this would massively boost the popularity of our product with our customers which are the farmers who not only want the best and latest IOT technology but something that they can use without trouble and understand clearly without having to learn the technical part:

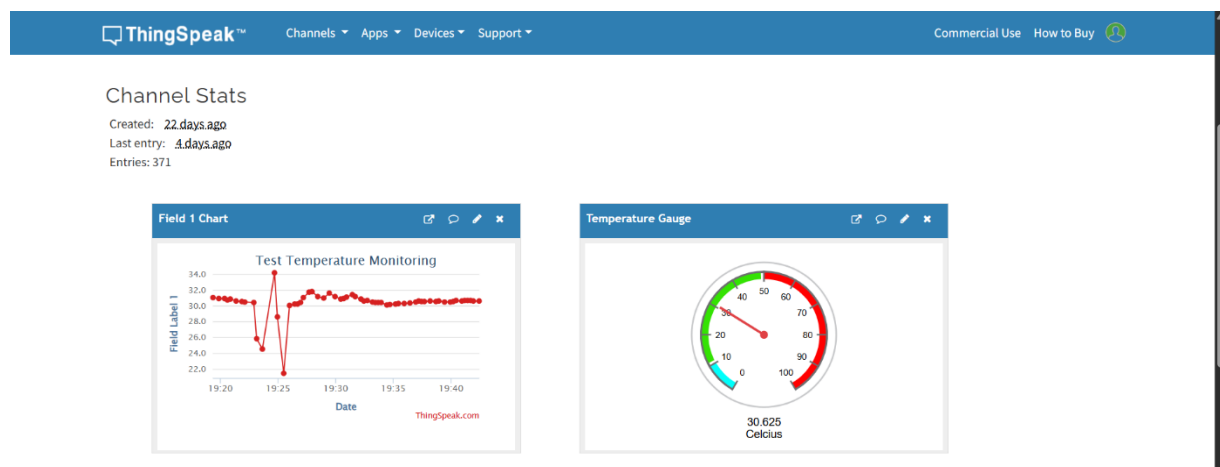


Figure 3.3.1: Desktop GUI of our ThingSpeak channel which allow live reading of temperature from our sensor and a full chart to track the temperature reading from the past which can be used for analysis

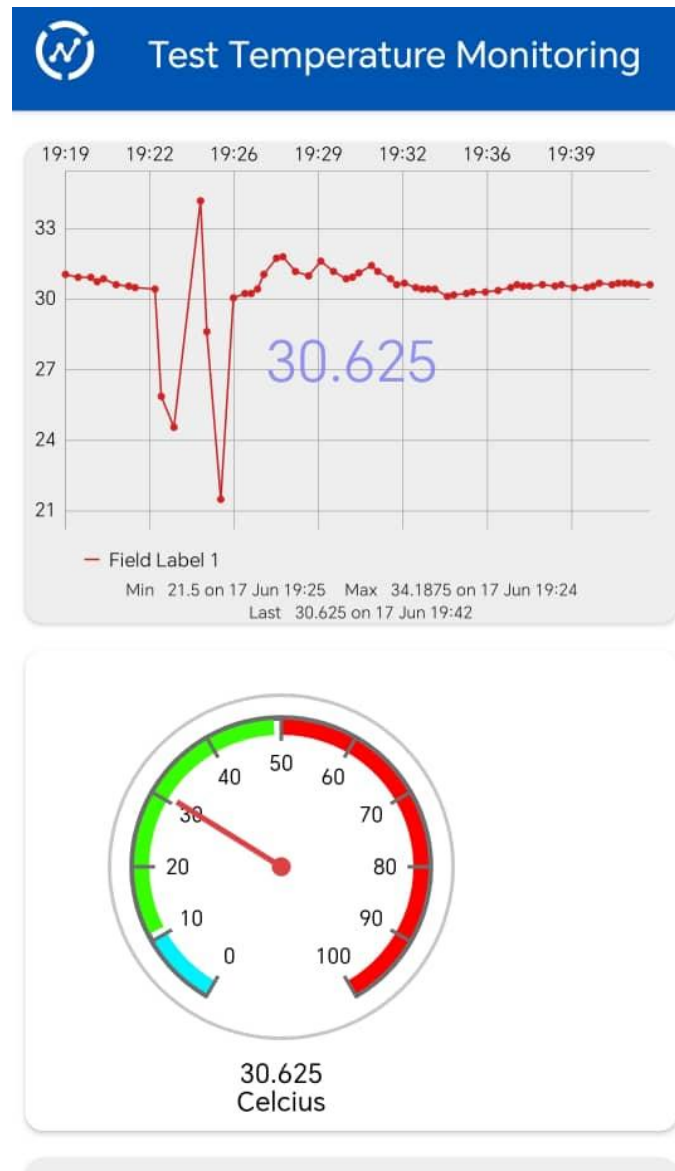


Figure 3.3.2: Our ThingSpeak channel on a remote portable device (Android) using the Thingview application for remote monitoring and live data

The code we created in order to test the ThingSpeak channel API linking and live monitoring in the initial stages:

```
// How to connect to ThingSpeak: This project connects to ThingSpeak using the
ESP8266 WiFi module.

#include <ESP8266WiFi.h>
#include <WiFiClient.h>
#include <ThingSpeak.h>

const char* ssid = "Your_SSID";          // Your Network SSID
const char* password = "Your_PASSWORD";  // Your Network Password
```

```

WiFiClient client;

unsigned long myChannelNumber = 2975330;
const char * myWriteAPIKey = "P7N4N8GGXILVWY9P";

int val;

void setup()
{
  Serial.begin(9600);
  WiFi.begin(ssid, password);

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected");

  // Seed random number generator using analog noise
  randomSeed(analogRead(A0));

  ThingSpeak.begin(client);
}

void loop()
{
  val = random(0, 101); // Random number between 0 and 100
  Serial.print("Temperature: ");
  Serial.print(val);
  Serial.println("*C");

  ThingSpeak.writeField(myChannelNumber, 1, val, myWriteAPIKey);

  delay(15000); // ThingSpeak requires at least 15s delay between writes
}

```

Instead of using the real sensor, we coded it so that a random number between 0 and 100 is generated. This is because if something goes wrong in this section of the code we can easily debug it compared to if we integrate everything at once which makes it more difficult to debug.

3.4 Blynk IOT

But after further considerations, we changed our decision to go with Blynk IOT instead of ThingSpeak, this is because Blynk provides a more user-friendly interface with time monitoring features similar to ThingSpeak, but most importantly, the ability to do real-time control is the main reason why we made the switch. This allows us to remotely control the solenoid valve in our system and track temperature and other important data directly from the Blynk website or mobile app, which is what our vision is for this product, rather than just a plain visualization of charts on a web dashboard. In addition, Blynk also offer better connectivity and more customizable widgets and automation options that would make our product stand out more and overall Blynk is just more suitable and more practical to be used in our system than ThingSpeak.

So, how does Blynk works, well Blynk simplifies IoT development by providing us with both the software (app, cloud, libraires) and infrastructure (servers, API, widgets) that are needed to make our system SMART and perfect to be used as a device for Internet of Things. The core components of Blynk includes we can use the Blynk library to communicate with the cloud, as the program can be connected to our sensors. In our case the ESP8266 microcontroller reads temperature via the DS18B20 temperature sensor and controls the relay which acts as a control device to activate the actuator which is the solenoid valve. Secondly, we can think of Blynk as a middleman between our hardware and the app as it manages everything in our system, including storing data and managing communications between web, microcontroller and actuator, creating an efficient closed loop control system. Blynk already have a ready-made interface where we can easily create dashboards using widgets such as buttons, sliders, graphs, gauges and more, as seen in the figure below:

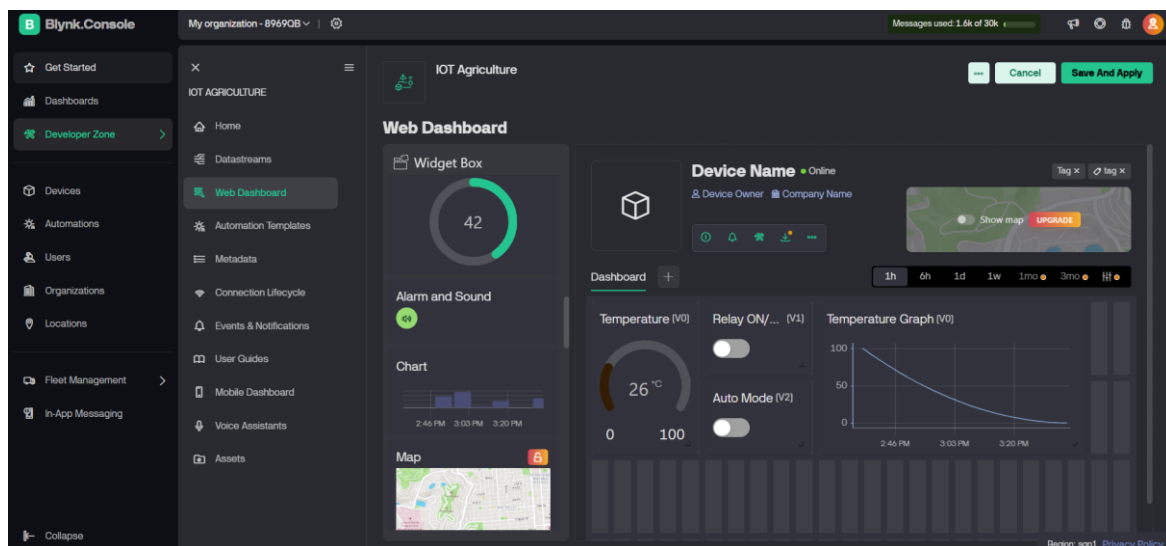


Figure 3.4.1: Customizable of widgets on Blynk

Setting up Blynk is also relatively simple, we first have to flash our ESP8266 with the code that includes Wi-Fi credentials (SSID and password of our network) and also our unique Auth Token that links our ESP8266 with our Blynk Cloud for our respective project:

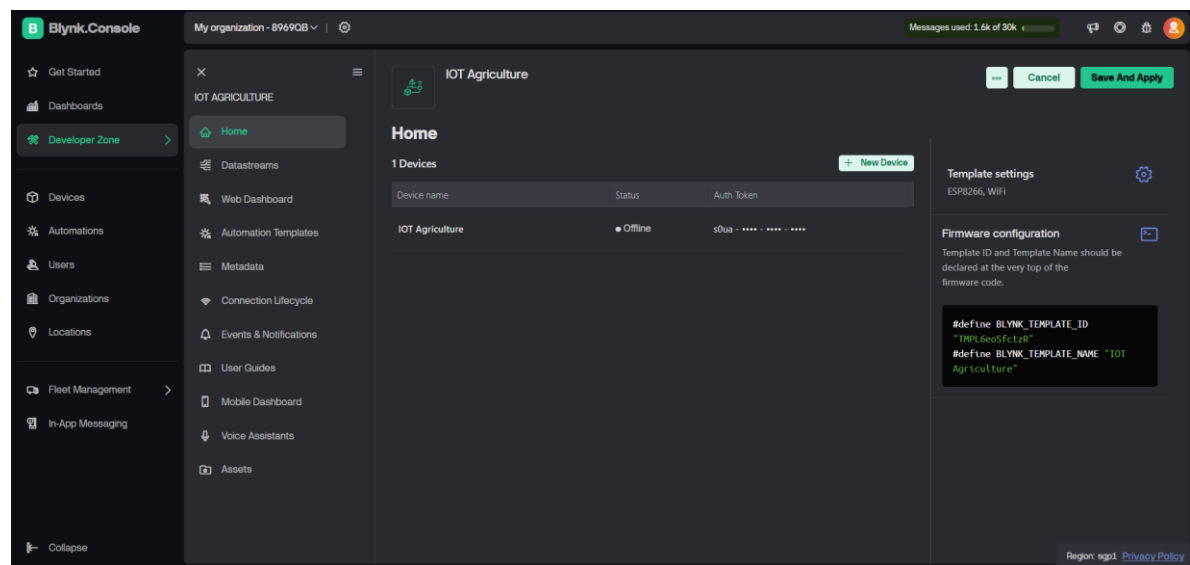


Figure 3.4.2: Auth Token and unique cloud address for Blynk Cloud

Once we linked our microcontroller with the Blynk Cloud, we must create pin data streams for each different type of data. In our case, we have 3 different types of data which are the Manual ON/OFF data (basically turning the valve ON and OFF manually), temperature data which is obtained from the DS18B20 sensor and also Auto mode ON/OFF data (basically, is the auto mode ON or OFF), hence we need to create 3 different pins for data stream in order to communicate with our devices:

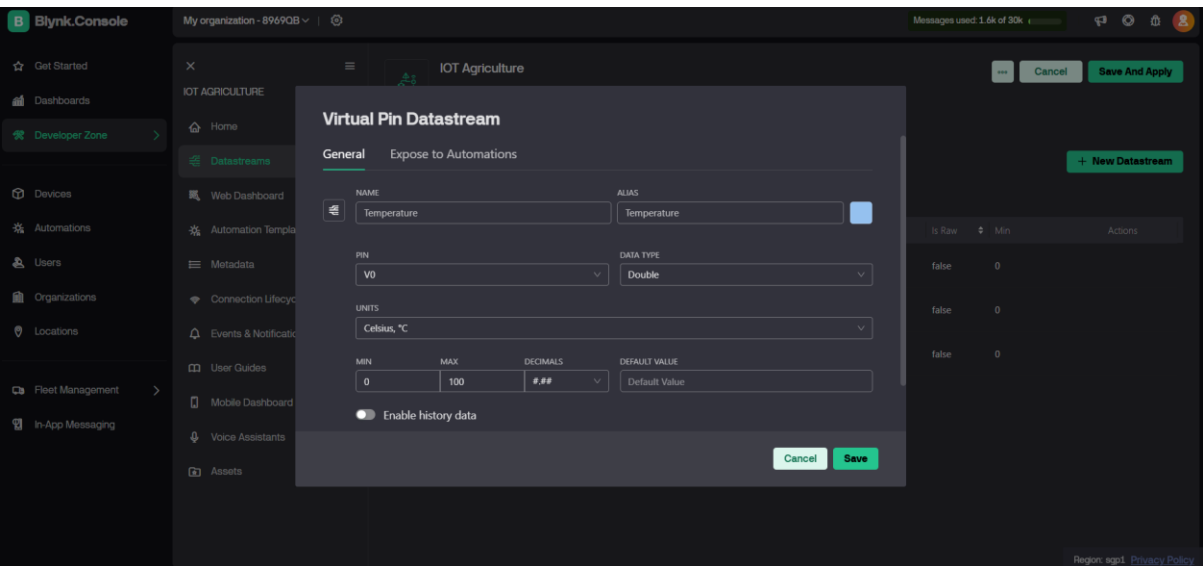


Figure 3.4.3: Creation of pin Data Stream for different types of data

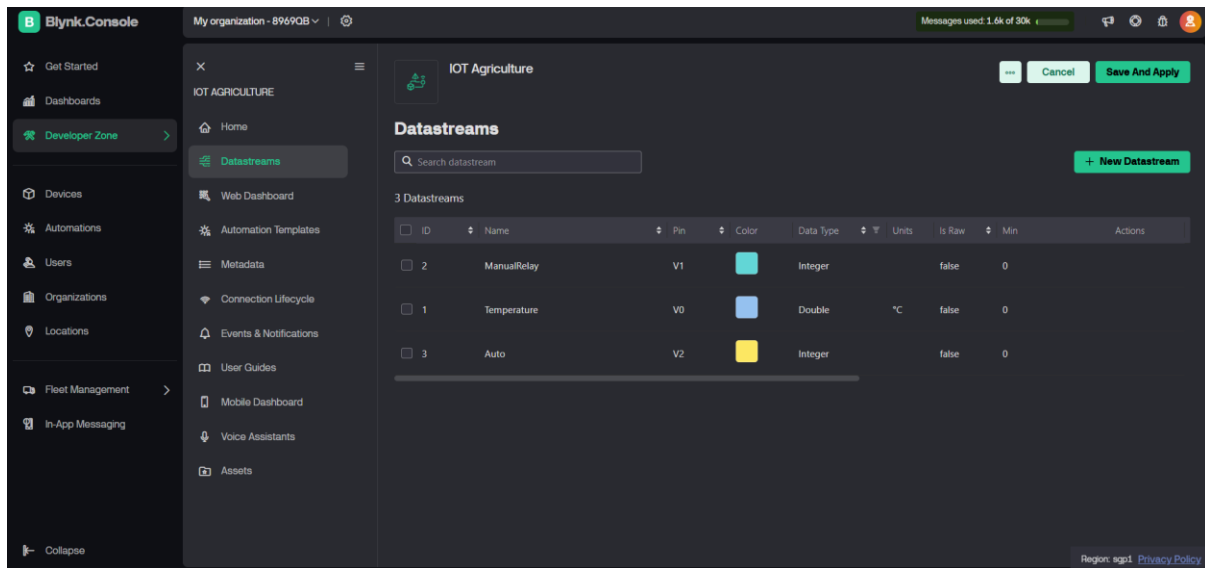


Figure 3.4.4: Our created pin Data Stream to be used for communication in Blynk IoT

Following this, we link the respective pins to our microcontroller via the code by using the library code of **Blynk.virtualWrite()**; in order to send data to the Blynk cloud, for example for our temperature data we used the code, **Blynk.virtualWrite(V0, tempC)**; in order to send the temperature data to the cloud of Blynk. On the other hand, the code `param.asInt()` is the code where Blynk sends data to the ESP8266, as in our case if the user manually turn ON the relay, Blynk will send a signal to the microcontroller and on this line of code, the ESP8266 will then turn ON the relay in order to activate the solenoid valve.

Other than the ease of setting it up and the amazing functionality of Blynk, it is also multi-platform supportive, meaning you can use it via the website on your computer or via the mobile app on your phone or tablet:

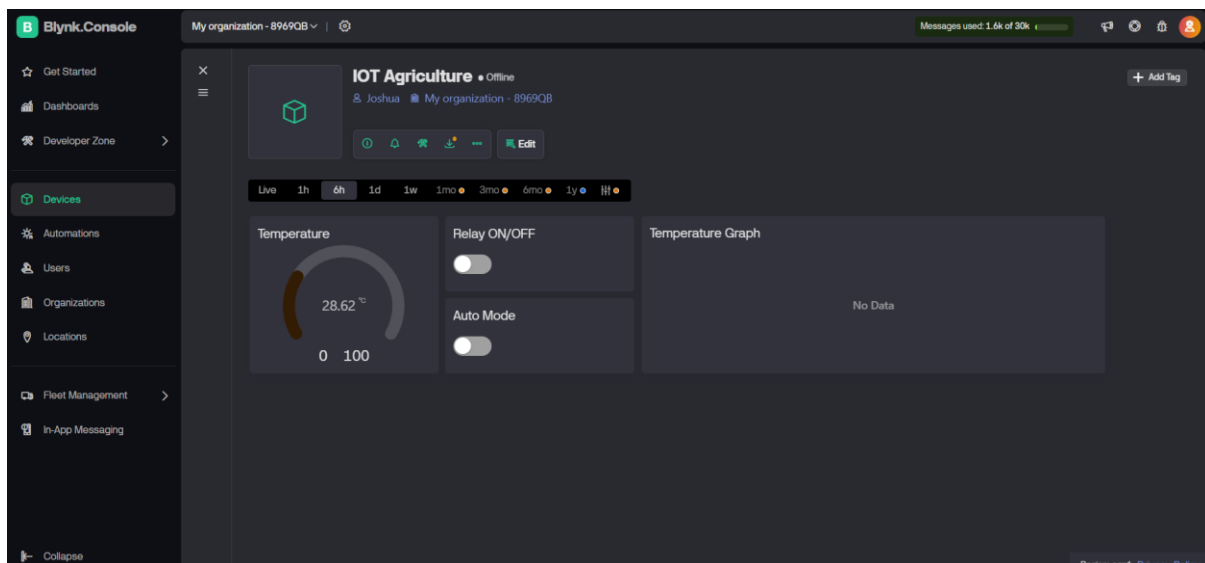


Figure 3.4.5: Dashboard of Blynk on the Blynk Website (PC)

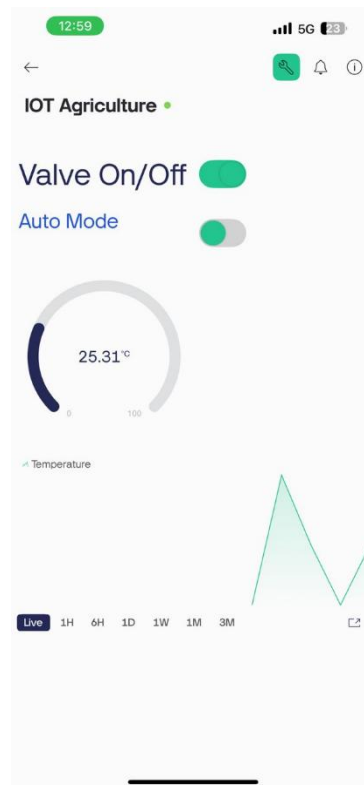


Figure 3.4.6: Dashboard of Blynk on the Blynk Application (Phone)

Besides this, if the user wanted to integrate with other software, Blynk can also do so, as it allow connection to software such as Blues, AWS, Myriota and more.

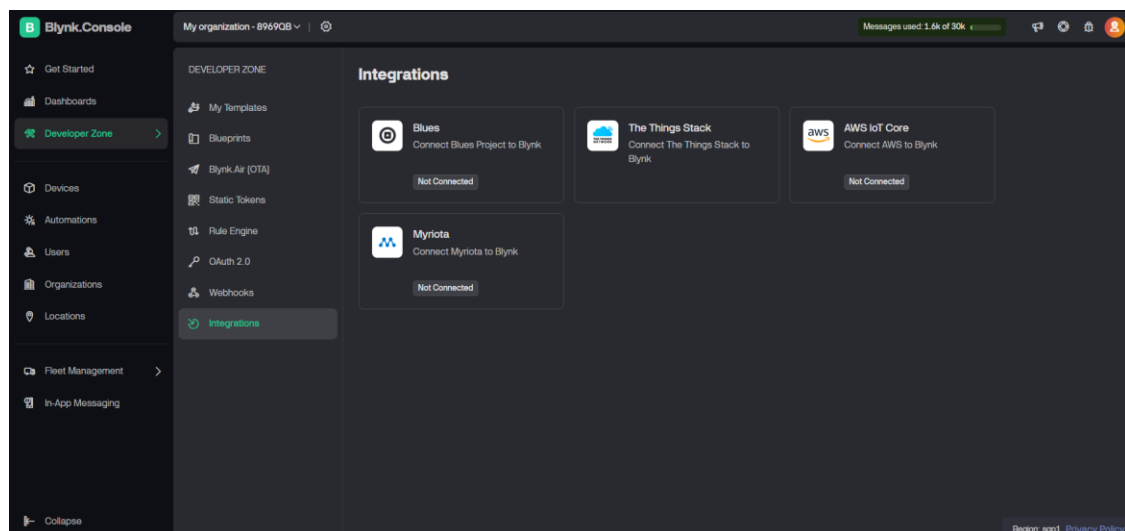


Figure 3.4.7: Integration of Blynk with other IoT software

It also has built-in blueprints to help user get an idea on how to get started and view ideas from other creators, making it a vibrant community to learn about how to create IoT projects tailored to users' own creativity and brilliance:

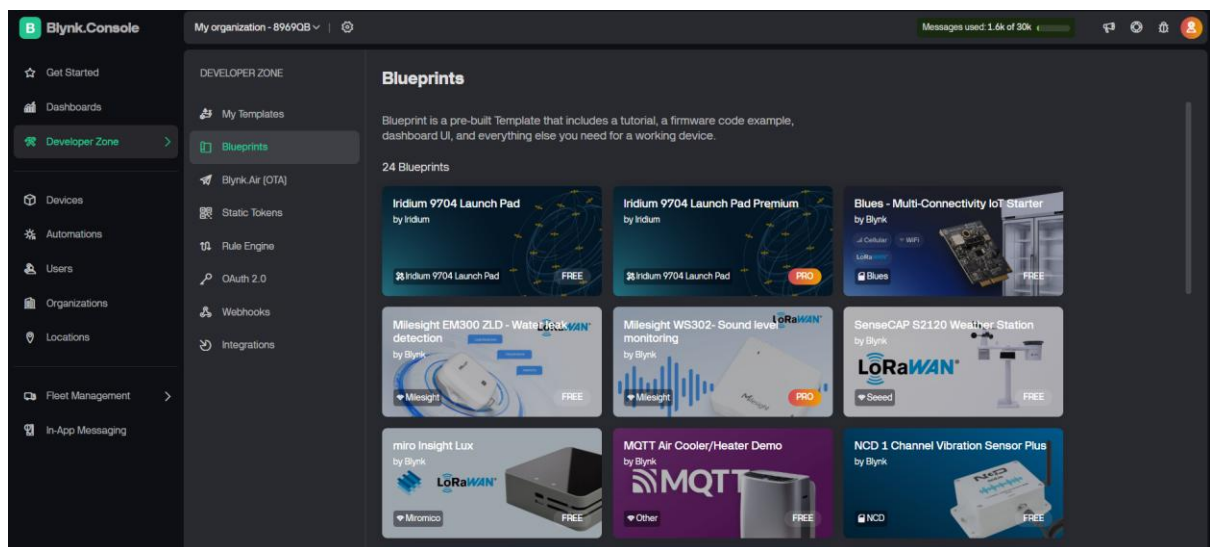


Figure 3.4.8: Built-in blueprints for Blynk IoT projects

Other than that, it also has a build in automation system, where user can schedule their own events, triggering certain actions on their IoT project on a specific state and time:

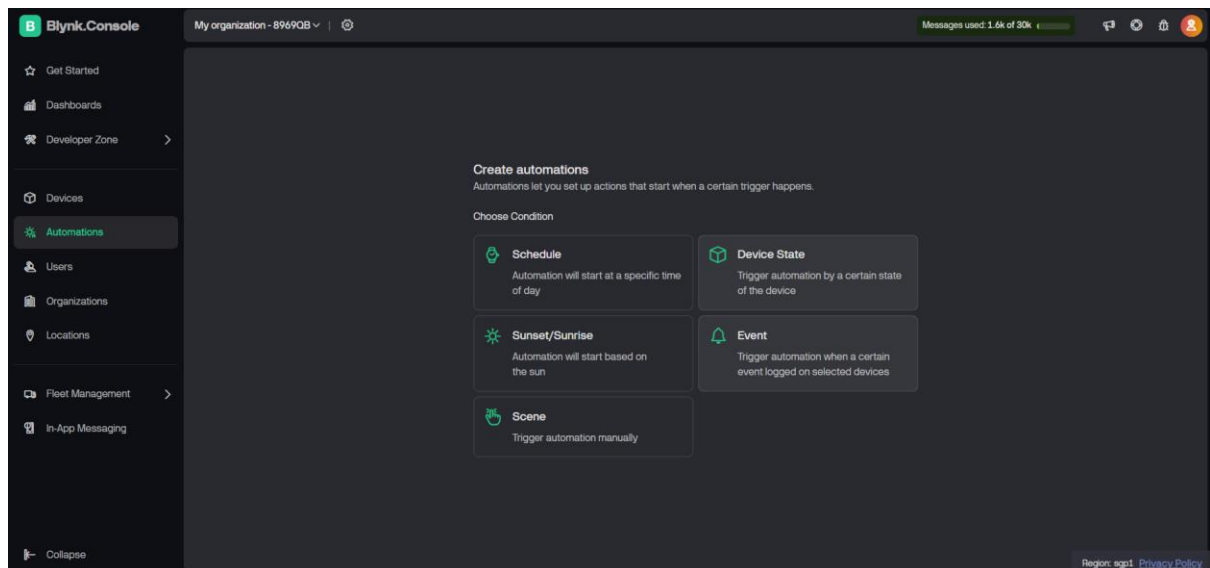


Figure 3.4.9: Automations that can be done to customize own IoT projects

And just like github and VS Code, it allows you to have multiple users for the same project, advocating for easy teamwork:

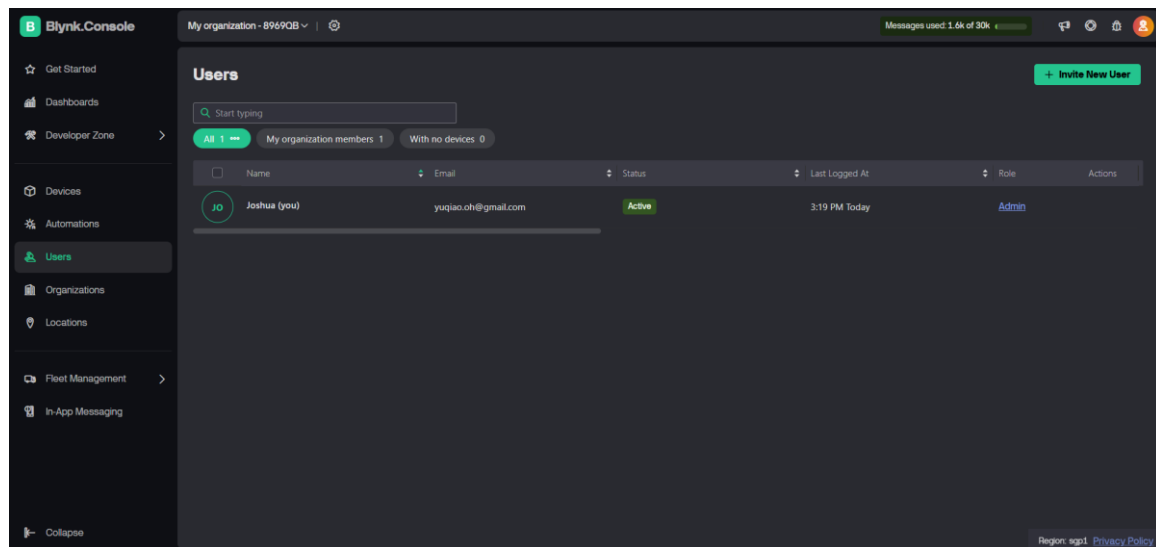


Figure 3.4.10: Features on collaborating with other like-minded people and friends

In the end, our Blynk dashboard basically tracks down the temperature data obtained from the ESP8266 microcontroller and sends data to the ESP8266 on the action of relay, whether it should be on Auto mode or Manual mode and if Manual mode whether it should be ON or OFF state, and our final dashboard for Blynk looks like this:

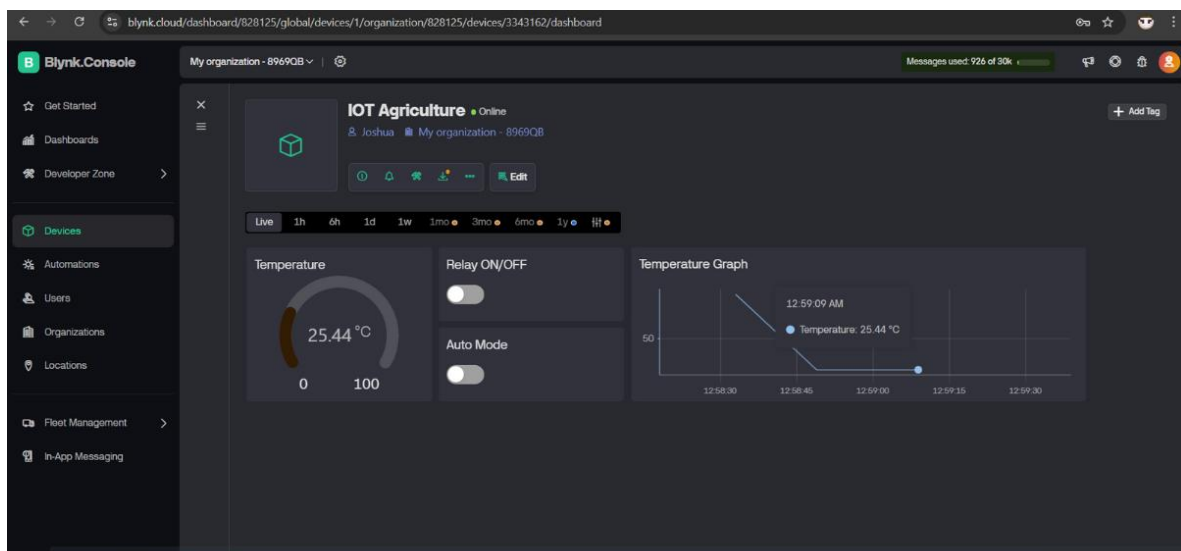


Figure 3.4.11: Final completed Blynk Dashboard for our automatic irrigation system

3.5 Components Selection for circuit

No.	Component Name	Function
1	NodeMCU ESP8266 Microcontroller	Processes sensor data and controls relay/valve operation
2	DS18B20 Temperature Sensor with protection module	Measures the temperature of water in the irrigation pipe
3	5V 1 Channel Relay Module	Acts as a switch to control solenoid valve based on ESP8266 signal
4	LCD Display Dual USB PWM Solar Panel Charge Controller Battery Regulator	Regulates the charging process from a solar panel or other input sources to lithium-ion batteries, ensuring safe and efficient charging while protecting against overcharge and discharge
5	Solenoid Valve (12V)	Controls actual water flow based on temperature thresholds
6	12V Rechargeable Batteries with holders	Powers the system in off-grid agricultural environments
7	Solar Panel	Charges battery using renewable energy in outdoor farm settings
8	Voltage Regulator (LM7805)	Steps down voltage to a safe level of 5V for ESP8266 and other components operation
9	Diode (1N4007)	Blocks the back-EMF generated by the solenoid valve when it is switched off and acts as a protection component for the relay
10	Half-size 400 holes Solderless Breadboard	Holds and connects all the electrical components together and acts as a foundational base for our overall circuit
11	Jumper Wires & Connectors	Links and connects all components electrically within the circuit
12	12 V DC Power Supply	Alternative power source which provides a source voltage of 12V for the operation of both the Microcontroller and Solenoid Valve

Table 3.5.1 List of component names and their respective functions

3.6 Schematic diagram/ circuit connection

The way we designed the circuit diagram is that we separated the whole entire circuit into 2 parts, which is the main power line which has 12 V voltage across and the 5V regulated power line. This is so that it is easier to work with and reduce the risk of accidentally shorting or overproviding voltage to components which are rated at different voltage levels. The layout of the circuit diagram can be seen below part-by-part along with the full circuit diagram when everything is connected:

Part 1 (5V regulated power line):

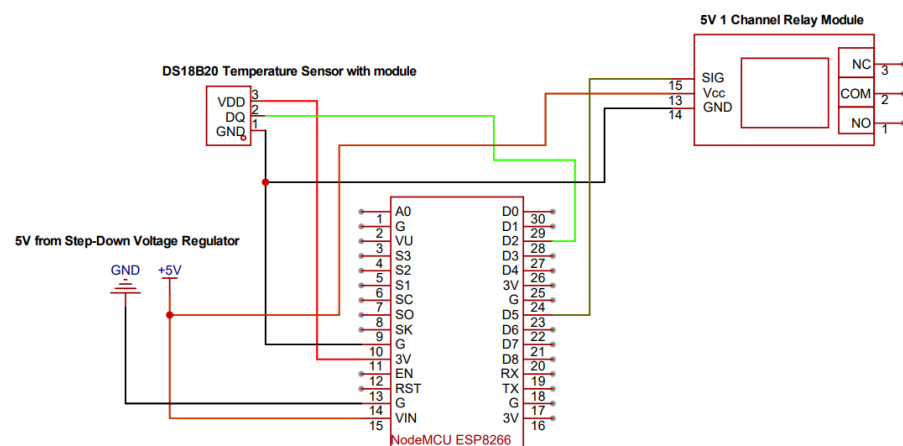


Figure 3.6.1: Circuit Connection of 5V Regulated Power Line

Part 2 (Main 12 V power line):

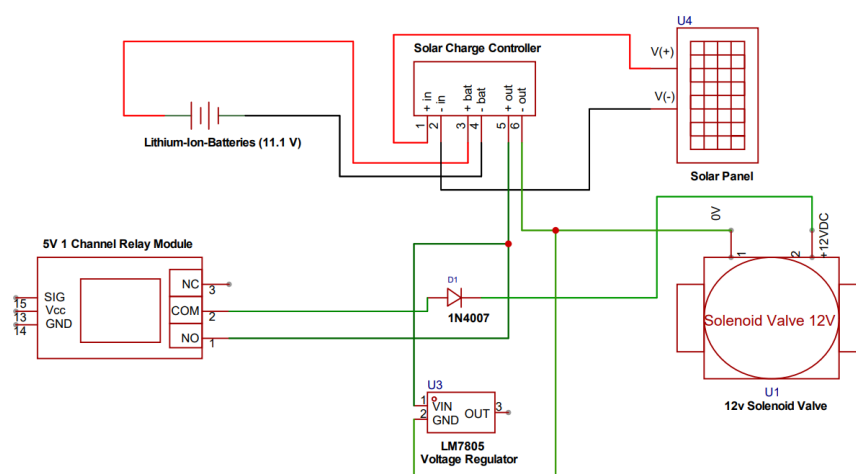


Figure 3.6.2: Circuit Connection of Main 12V Power Line

Entire circuit schematic diagram of our product:

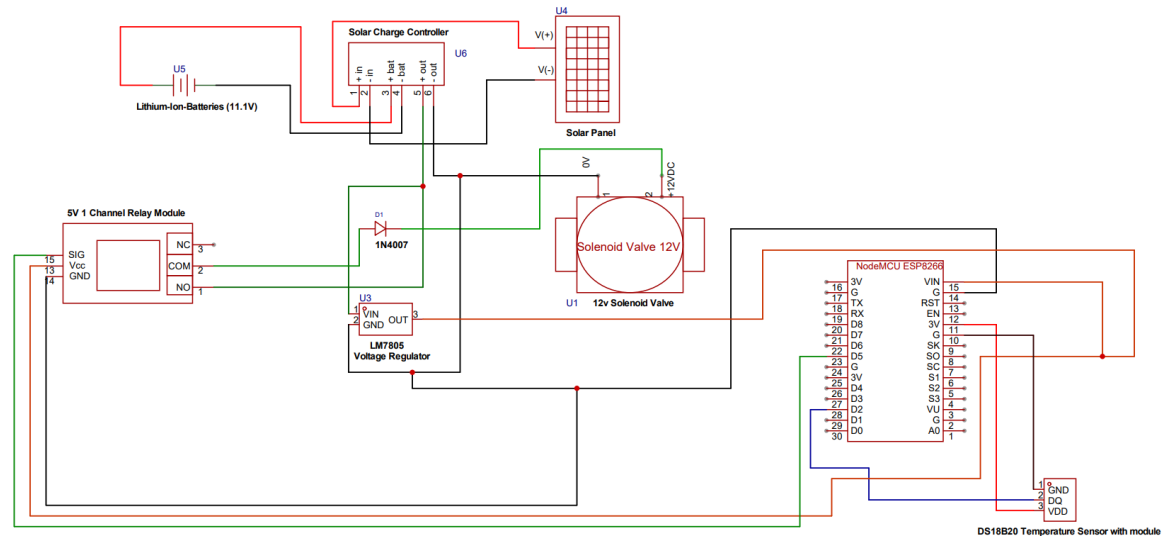


Figure 3.6.3: Full Circuit Schematic of our Automatic Irrigation System

Part 1 (5V regulated power line):

Based on Figure 3.6.1 above, the 5V supply is used to power the microcontroller ESP8266, the temperature sensor DS18B20 and the relay module. This 5V which comes from a step-down voltage regulator is connected to the VIN of the microcontroller and the GND of the voltage regulator is connected to the GND of the other modules. The DS18B20 temperature sensor's VDD is connected to the 3V output from the ESP8266 as if we connect it directly to the 5V line it may damage the DS18B20 module via overheating, its GND to the common ground and the Data line is connected to the D2 pin of the microcontroller to get the data for the temperature. The microcontroller will then read the values of the temperature from the temperature sensor and will process these readings. Additionally, the relay module's VCC goes through the same 5V, its GND goes to the ground of the system the signal pin of the relay goes to the pin D5 of the microcontroller, such that when the microcontroller sends a signal to the relay, it switches ON or OFF and hence, this allows the microcontroller to process the readings of the temperature from the temperature sensor and send signals to the relay for it to activate the solenoid valve when needed. Initially we wanted to add the I2C LiquidCrystal LCD display to our system, but after further implementations we decided to remove it entirely from the system, as we already link our system via a IOT software "Blynk" which can display all the necessary information including connectivity, temperature data via graphs and other important information about the whole system, hence eliminating the LCD display not only help avoid redundancy but also reduces power consumption, thereby extending the battery life and improving the overall efficiency of the system in general.

Part 2 (Main 12 V power line):

Based on Figure 3.6.2 above, this part of the circuit represents the 12 V power that is needed to control the solenoid valve. The solar panel is connected to a solar charge controller which will charge the 11.1 V battery and will ensure a safe power output. The solar charge controller with the almost 12 V output (precisely 11.1 V output) will then power the solenoid valve through the relay which acts as a switch that is controlled by the microcontroller. The COM terminal of the relay module is connected to the power line 12 V and the NO which is the Normally Open terminal of the relay is connected to one side of the solenoid valve. The other wire of the solenoid valve is connected to the GND and through this way, when the microcontroller sends a signal to the relay, there is the existence of a closed connection between the COM and the NO such that the 12 V is allowed to flow through the solenoid valve, activating it and hence it is turned ON. The 1N4007 diode, which allows the current to flow in one direction only, is placed across the solenoid connection. This diode is to prevent backflow current also known as back EMF from the solenoid valve which can damage the relay. Moreover, the LM7805 voltage regulator regulates and step-down the voltage from the 12V source to 5V which is needed to power the components from part 1. In simple words, this part of the circuitry basically consists of components that require 12V to power them up and how the main power source of 12V (precisely 11.1 V) interact with the smaller 5V components from our system in general.

Part 3 (Entire circuit schematic diagram):

Based on Figure 3.6.3 above, the system starts by using a solar panel connected to the solar charge controller which charges the 11.1V battery and provides 12 V (11.1 V precisely) output to power the system. It also makes it so that it is very easy to link with external power not just solar panels, as user can easily swap out the wires if they wanted to use say a wall plug instead, so this charge controller provides user with better modularity and possibility of limitless options when it comes to powering the whole system. This 12 V will then go to the relay that acts as a switch for it to control the solenoid valve while also the 12 V is sent to the voltage regulator LM7805 to change the 12 V to a 5V supply which is used to power the microcontroller and the relay module. The microcontroller which is connected to the DS18B20 temperature sensor has 3 pins out of which one is connected to power the microcontroller, the other is connected to the ground while the last pin sends the temperature data into the pin of the microcontroller. The relay module's VCC (5V) is from the voltage regulator, its GND goes to the common ground, and the other pin is connected to the microcontroller's pin. The COM terminal of the relay is connected to the 12 V supply, and the NO (Normally Open) terminal is connected to the positive side of the solenoid valve while the negative side of the solenoid valve goes to the ground. The diode (1N4007) which is across the solenoid protects the circuit from back EMF when the solenoid valve is OFF and it also allows the current to flow in one direction only. The operating mode is such that as the solar panel charges the battery and supplies the circuit, the voltage regulator allows a 5V supply to the components and the microcontroller reads the temperature data from the DS18B20 temperature sensor. When the temperature goes above the set point or when the user directly gives signals to the

microcontroller via IOT (Blynk), the microcontroller sends a signal to the relay which will close it, allowing 12 V and current to flow through the solenoid valve and letting the water to pass. On the other hand, when the relay turns OFF and the solenoid valve closes.

3.7 Software selection for 3D modelling and printing

Since we were new to 3D modelling and had no prior experience in this field, selecting the right software was a crucial step in our 3D modelling and printing process. With many options available online, our primary goal was to find a platform that is user-friendly and well-suited for beginners. After conducting research and seeking advice from peers with relevant experience, we identified Onshape and Onshape as two promising options. To make an informed decision, we compared both based on ease of use, accessibility and learning curve. Feedback from experienced users and our own exploration indicated that Onshape offered a more intuitive and beginner-friendly interface. As a result, we decided to use Onshape for our 3D CAD modelling and printing tasks.

3.8 3D prototype design of casing

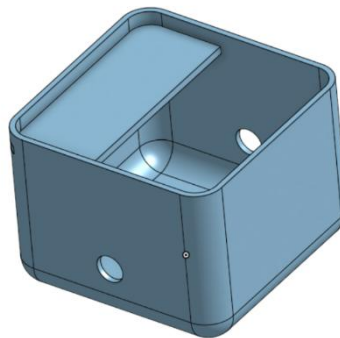


Figure 3.8.1 Container

After discussion with our group members, it was determined that the container must meet specific requirements to accommodate a solenoid valve, water piping, electronic components and a solar panel. Initially, we designed the container with dimension of 30cm x 30 cm x 30cm to fulfil these needs. However, after inspecting the 3D printing machine, we discovered that it could only support a maximum print size of 25cm x 25cm. As a result, we revised our design and reduce the container dimensions to 20cm x 20 cm x 15cm as illustrated in Figure 2.1 and a separate cover was designed to house the solar panel. To allow the water pipe to pass through the container and connect to the solenoid valve inside, a 25mm diameter hole was added to the side of the container. As shown in figure, a platform with dimensions 8cm x 20cm was also designed within the container to securely place the electronic components.

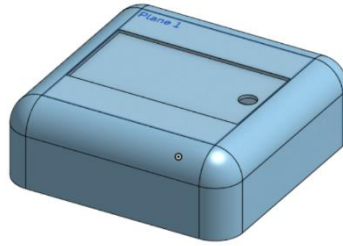


Figure 3.8.2 Container Cover

A cover with dimensions 20cm x 20cm x 7cm was designed for the container as shown in Figure 2.8.2 to facilitate easy maintenance and inspection of internal components, such as the connection between the pipe and the solenoid valve, as well as the electric circuit. The cover also serves a functional purpose related to the solar panel, which needs to be positioned outside the container for optimal sunlight exposure. To accommodate for this, a 15cm x 8cm concave section was incorporated into the surface of the cover, along with a hole to allow wiring from the solar panel to pass through and connect to the microcontroller which is housed inside the container.

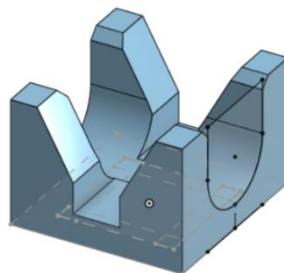


Figure 3.8.3 Solenoid Valve Holder

Our valve is made from brass, which is an alloy of high mass. To accommodate for this aspect of the valve, we have also designed a solenoid valve holder to ensure that the solenoid valve can be securely and stably positioned within the container, to prevent it from rocking horizontally. The holder was designed with dimensions of 11cm x 11cm x 7.5cm as shown in Figure 2.8.3, and its structured was tailored to fit both the container layout and the dimensions of the solenoid valve. Specific features were included in the design to allow for easy placement and alignment, ensuring a snug fit that minimizes the movement of solenoid valve during operation.

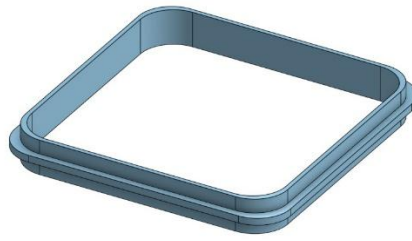


Figure 3.8.4 Connector

The connector functions as a snap-fit locking interface that secures the container body and its cover through geometric interlocking. The design incorporates stepped edges and circumferential grooves along the outer wall, which engage with corresponding recesses on the mating component when assembled. This configuration ensures precise alignment during closure while providing a mechanical resistance that prevents unintentional displacement of the cover under normal handling conditions. The snap-fit mechanism relies on localized elastic deformation of the material, allowing repeated assembly and disassembly without compromising structural integrity. In addition, the continuous contact along the interlocking surfaces enhances stability and, when combined with suitable tolerances or sealing elements, may contribute to improved sealing performance against external contaminants. Compared with fastening methods such as screws, clips, or adhesives, the snap-fit design offers advantages in terms of manufacturing simplicity, reduced assembly time, and reusability, making it a cost-effective and efficient solution for container-cover integration.

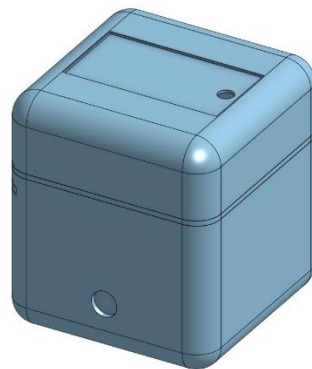


Figure 3.8.5 Container after combined

The assembly consists of a container body and a detachable cover joined through an integrated snap-fit connector. The interlocking features along the midsection of the enclosure allow the two components to be securely engaged without the need for external fasteners, thereby ensuring stability during operation while still permitting intentional disassembly when required. The enclosure incorporates circular openings on selected surfaces, which serve as pipe inlets and outlets. These openings are designed to allow piping to pass through the housing and establish a direct connection with a solenoid valve positioned within the container. This arrangement enables controlled fluid flow through the system while maintaining a compact and

protected assembly. The smooth, filleted edges of the enclosure ergonomics and reduce stress concentrations, contributing to both safety and durability.

4.0 Purchasing/ Cost Optimization

4.1 Strategies & Impacts

Demand & Suitability Forecasting

Upon being handed our project objectives and understanding the task, we made good use of online resources such as YouTube, ChatGPT to provide us with suggestions and comparisons between components and materials. After careful evaluation and elimination, we settled on a list of materials that would allow for our project to operate at a high-output, low-cost manner, such that the project deliverables are directly addressed whilst maintaining the practicality of the proposal.

Prompt Purchasing of Critical Components & Materials

To avoid slacking from the timeline, we started purchasing components on week 4 of the project. Considering that some of the components had to be purchased online, that they would also require a certain duration for shipping, we handled online purchases instantly after a decision is made amongst our team members. As a result, we can measure the dimensions required for the 3D modelling stage of our project whilst keeping up with our timeline effectively.

Prioritising Online Shopping over Retail Shopping

We have discovered that most components are sold relatively cheaper online compared to buying physical. Therefore, we utilised the online platform, Shopee to obtain most of the components used for our project. Besides, we made sure to apply any coupon code to further reduce the cost borne by our team. Not only did we save on financial cost, but we also saved on time cost as we do not have to look for the components in person.

4.2 Recommendations

Out of the three strategies that we implemented, to is highly recommended that future projects prioritise demand & suitability forecasting to determine the best course of action to produce the best product. By implementing the forecasts correctly, reworks and incompatibility issues can be reduced significantly, leading to a smoother delivery of deliverables.

4.3 Components Cost

The following table shows the list of components we used during our development phase and their respective costs of acquisition.

Equipment/ Services	Cost Per Unit (RM)	Unit	Total Cost (equipment) (RM)
DS18B20 (temperature sensors)	11.00	2	22.00
DS18B20 Sensor Module	7.99	1	7.99
Two Way Brass Solenoid valve	54.76	1	54.76
Jumper Wires, wires	Provided	40	Provided
Breadboard	Provided	1	Provided
LCD with I2C module	Provided	1	Provided
3D printing with materials	Provided	1	Provided
DC power supply	Provided	1	Provided
Relay Module	Provided	1	Provided
Voltage Step Down (Power module)	5.19	2	10.38
NodeMCU ESP8266	Provided	1	Provided
USB to Micro USB cable	6.90	1	6.90
Piping, connectors and joints (PVC) set	14.30	1	14.30
Solar Panel	Provided	1	Provided
4.7 k Ω resistor	Provided	2	Provided
Diode (1N4007)	Provided	1	Provided
Copper wires cable	Provided	1	Provided
3.7V Rechargeable Batteries	3.73	3	11.19
Battery Holders	8.39	1	8.39
TOTAL (RM)			135.91

Table 4.3.1 Estimate of component cost during product development

Upon finalising the product, we have concluded and calculated the total estimated cost for reproduction at a small scale. Below is the table of costs:

Equipment/Services/Materials	Cost Per Unit (RM)	Unit	Total Cost (equipment) (RM)
DS18B20 (temperature sensors)	11	1	11.00
DS18B20 Sensor Module	7.99	1	7.99
Two Way Brass Solenoid Valve	54.76	1	54.76
PVC Untinned Single Strand Copper Wire (per meter)	0.75	1	0.75
Jumper Wires (20 per pack)	2.5	1	2.50
Breadboard (84mmx56mm)	3.5	1	3.50
3D printing with materials	30	1	30.00
5V 1 Channel Relay Module	1.65	1	1.65
Voltage Step Down	5.19	1	5.19
NodeMCU ESP8266	12.65	1	12.65
USB-A to Micro USB cable	6.9	1	6.90
Piping, connectors and joints (PVC) set	14.3	1	14.30
Diode (1N4007)	0.5	1	0.50
3.7V Rechargeable Batteries	3.73	3	11.19
Battery Holders	8.39	1	8.39
TOTAL (RM)			171.27

Table 4.3.2 Estimate cost per unit for reproduction

5.0 Fee Summary

5.1 Introduction

The fee summary of this agriculture automation project provides us with an opportunity to get an overview of the financial status and budgeting of the project as of 19th June 2025. This include a detailed breakdown of the project's budget, actual costs incurred to date and a variance analysis to highlight whether if there are any deviations from the original financial plan or are we on track. Furthermore, this section also gives us a projection of future costs and expenses, this information can then be used to predict and outline the expected final price of the product itself. The main objective of this summary is to provide an update and clear insights on the financial performance of the project, identify whether any adjustments needed to be done or any cost-related concerns that need to be taken care of in order to meet the main financial goal for this project and also recommendations to ensure the project stay within the budget while achieving the intended outcomes of the project.

5.2 Budget overview/Planned Budget

Total Project Budget: RM 150

Phase Breakdown:

- Planning/Conceive: RM 5
- Design: RM 5
- Implementation: RM 130
- Operation/Testing and Launching: RM10

5.3 Actual Cost Incurred (as of 14th August 2025)

- Planning/Conceive: RM 0
- Design: RM 0
- Implementation: RM 145
- Operation/Testing and Launching: RM 0

5.4 Allocation breakdown (Planned Budget vs Actual expenditures)

Table of breakdown between Planned Budget vs. Actual Expenditures:

Phase	Planned Budget (RM)	Actual Expenditures (RM)
Planning/Conceive	5.00	0.00
Design	5.00	0.00
Implementation	130.00	145.00
Operation/Testing and Launching	10.00	0.00

Table 5.4.1 Comparison of planned budget and actual expenditure

5.5 Variance Analysis

Variance analysis is a technique used in project management to compare your planned budget figures with the actual figures to determine whether the project is financially on track. Using the formula below, we have done the full variance analysis:

$$\text{Variance} = \text{Actual cost} - \text{Budgeted Cost}$$

$$\text{Percentage Variance} = \frac{\text{Variance}}{\text{Budgeted Cost}} \times 100\%$$

Phase	Budget (RM)	Actual Cost (RM)	Variance (RM)	% Variance	Remarks
Planning/Conceive	5.00	0.00	-5.00	-100%	Completed within budget
Design	5.00	0.00	-5.00	-100%	Completed within budget
Implementation	130.00	145.00	15.00	11.54%	Fully updated upon finalisation
Operation/Testing and Launching	10.00	0.00	-10.00	-100%	Testing only consisted of complimentary or low cost.

Table 5.4.2 Variance analysis

The actual cost of the project strictly controlled to adhere to our budget, reflecting effective and efficient financial planning and a systematic approach in cost control measures, as well as during the design phase. Variance analysis from original budget shows that overall, we have done well in closely aligning the spending with the original budget.

All phases (Conceive, Design, Implement and Operation) showed positive variance, proving that our team members have managed the resources well and responsibly and made strategic decisions according to the limits set by the budget without compromising the quality of the product or time factor.

5.6 Expected Final price of product

To calculate the expected final price of the product, we use the formula

$$\text{Expected Final Price} = \text{Actual Cost Incurred} + \text{Projected remaining Costs}$$

Actual cost of reproduction: RM 172.00

Projected remaining Costs: RM 40.00

$$\text{Expected Final Price} = \text{RM}172.00 + \text{RM}40.00 = \text{RM}212.00$$

We also need to add the additional cost provided by supplier (Distributions): RM 300

$$\text{True Expected Final Price} = \text{RM}212.00 + \text{RM}300.00 = \text{RM}512.00$$

5.7 Profit Margins/Final Selling Price

To calculate the final selling price for the market:

$$\text{Selling Price} = \text{Expected Final Cost} + (\text{Expected Final Cost} \times \text{Profit Margin})$$

We targeted a profit margin of 20% for our product, due to several factors. One of which is sustainable growth and reinvestment. The profit gained can be used in redevelopment and optimisation of our product, and investing into better infrastructure to achieve a better, more desired result. The profit margin set is also suitable for our scale, as we target to only operate this project at a small scale, with potential of growth in the future as we improve over time.

Profit margin/gain = 20%, so:

$$\text{Selling Price} = \text{RM}512 + (\text{RM}512 \times 0.20) = \text{RM}614.40$$

6.0 Reliability and durability considerations

6.1 3D modelling of casing

To achieve the desired level of reliability and durability, ABS was selected as the material for our 3D model printing. This choice was made based on ABS's superior performance in terms of sunlight exposure resistance, water resistance and mechanical durability, aligning well with the environmental and functional requirements of our project. To further enhance structural strength and impact resistance, the casing was designed with a wall thickness of 5mm. Additionally, a removable cover was incorporate into the design to facilitate easy maintenance of internal components. For repeated access, the cover will be secured using durable hinges or clips, ensuring long-term usability without compromising the integrity of the enclosure.

6.2 Circuit

Durability and reliability were important considerations in the circuit design, especially when it is used for outdoor farming. For instance, all the components such as the ESP8266 and the relay module are placed in a waterproof box to protect them from any weather conditions. The temperature sensor also being waterproof can be used safely even if it gets wet. Instead of adding a pull-up resistor for the sensor operation, we transitioned to a more reliable sensor module due to the challenges we faced, which is mentioned in the 7.0 Challenges section of our report. The main power source is a lithium-ion rechargeable battery pack that is rechargeable depending on the user's intention.

6.3 Other components

Other components which include the piping, joints and connectors are mostly made of sturdy materials for example PVC (polyvinyl chloride) which is widely one of the best materials against corrosion, chemical degradation and environmental stress, all of which are crucial especially in the agriculture industry, where involvements of chemical use or extreme temperature cases could be a factor. This makes PVC a great choice, as the material is not only lightweight, but it is very durable against internal pressure due to water pressure without cracking or deforming over time. The connection using the connectors and joints are also leak-proof and will be properly installed using a PVC connector glue. But overall, the system has long-term reliability, as we designed the system to suit for factors like UV exposure, temperature fluctuations, mechanical stress, water exposure and etc without compromising on user-friendliness and ergonomic. For example, the joints and connectors made it so that it is very easy to connect the system with the main LDPE irrigation Polypipe used in the agriculture industry:



Figure 6.3.1 PVC Converter (20mm to 25mm)

7.0 Development challenges

7.1 3D Printing

For the most part, 3D printing was successful and all components were printed close to desired proportions and appearance.

However, our inexperience caused us to wrongly measure the hole of the box according to the diameter of the PVC pipes. Apparently, we designed the hole to be 2.5mm in diameter, where our outer pipes are 2.8mm in outer diameter in actual. In other words, we did not account for the 1.5mm wall thickness of the piping.

To account for this, using the resources of our stakeholder, Sunway University, we had to sand our 3d printed casing using a sanding tool and we had to sacrifice some structural integrity of our 3d printing casing, introducing a potentially weak spot at the hole where we manually largen the hole. Minor cracks on the casing can be spotted if zoomed in properly.



Figure 7.1.1 Sanding of hole in waterproof casing

Aside from this, our initial design did not account for a locked cover on top of the container. To counteract this, we consulted experienced 3D printing personnel and discussed amongst our 3D printing representatives to come up with a solution to allow both the cover and the container body itself to be intact (locked) without the use of adhesives.

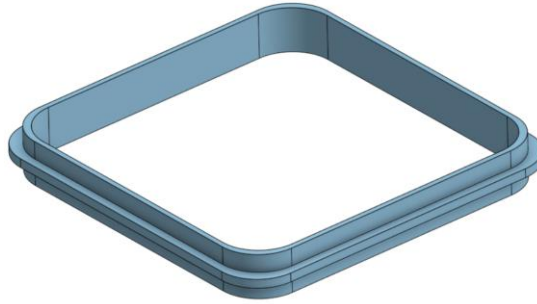


Figure 7.1.2 Image showing our solution for locking the container and its cover

7.2 DS18B20 Sensor Operation

The DS18B20 sensor is a one-wire sensor designed to operate scenarios where immersion in liquid is required due to its waterproof design characteristics.

However, the modularity of the sensor is limited and can be said to have room for improvement. Most of the issues we had when working with the DS18B20 include the error message -127°C which is said to be a communication failure error message between the sensor and the ESP8266 microcontroller unit. This error message is a default error code returned by the DallasTemperature library in the code for the proper operation of the DS18B20 sensor.

We suspected a few probable causes of this error message. One of which is intermittent connections and loose wiring. The use of many jumper cables may be the main culprit of this issue. Namely jumper cables become loose over time, and jumper wires can be faulty if not tested for connectivity before using it.

At times, the error code resolves by itself without knowing the reason behind. So, we believe that the DS18B20 is not too reliable in terms of reliability, however we had to use it due to its waterproofing characteristics.

In the end, we decided upon purchasing a designated module designed around the DS18B20 to improve the reliability of our product and reduce our slack on the project schedule.

7.3 Components troubleshooting

Having believed the DS18B20 malfunctioned in a way, we soldered everything just to find the -127°C error message still appeared. In one careless attempt to reupload the code to the microcontroller, we made a mistake by not disconnecting the microcontroller with its 5V V_{in} power source. In the end, our voltage step down converter had voltage conflict issues such that it smoked and became faulty in the end. Not only that, our microcontroller micro-USB port also became faulty in a way that we could not upload newer code onto the microcontroller. Sensor malfunction was the biggest issue we faced during our whole development.

Besides, the use of solenoids in solenoid valve, upon turning off, introduced back-EMF to our circuit in a way that the relay was continuously switched on and off in a rapid way. We included a diode in series with the relay to prevent the backflow of current to the relay under the induced back-EMF.

Additionally, two power modules of the same model and specifications were broken as we develop our product. We suggest it might be due to the inability to handle high voltage spikes as it was only rated 7-12V input. In the end, we decided upon the use of an LM7805 IC would suffice for our usage. However, the LM7805 is highly inefficient as it draws a lot of power and heats up upon long hours of usage. A better voltage converter should be opted for to better improve the energy efficiency of the device.

7.4 Power source

Initially, we planned for the system to be powered by a rechargeable battery connected to the solar panel for portability and efficiency. However, due to the highly complex regulation & configuration of the solar panel for it to work with our system, we decided to go upon using a lithium-ion battery combination as power source for our valve control system, with the option of switching to a 240VAC-12VDC power adaptor by using a fast-wire connector as shown in the image below. Such arrangement allows for flexible switching of power sources, giving our users multiple options to power the system.



Figure 7.4.1 Fast wire connector used in our system

7.5 Leakage from pipes

To secure our connections from water damage, we made sure to reduce leakage to a minimum by using plumbing thread seal tape.



Figure 7.5.1 Plumbing tape

For the DS18B20 sensor, we drilled a hole into the cap of a PVC connector and placed the sensor deep into the pipes such that it reaches the water. This was our biggest challenge as the hole we drilled for the sensor left a large gap. To tackle this issue, we used hot glue, PVC glue and white tack altogether and made layers of adhesive to accommodate for the gap.

To test for leaking, we first made sure of the area of testing was dry to touch and wrapped a water-absorbing material, tissue paper around the suspected areas of leakage.

In the end, our strategy was proven a success as no leaking was observed.

8.0 Implementation

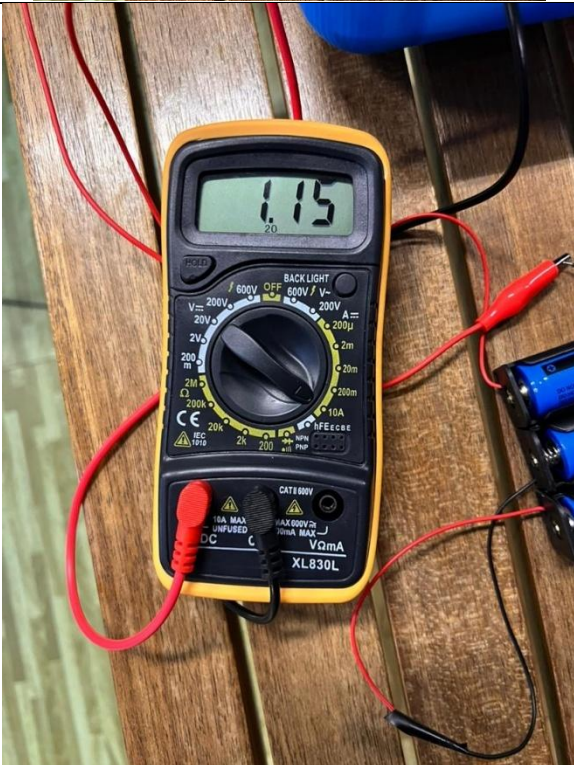
8.1 Power

We ran several tests on determining the power consumption of the system using 2 different sources, which are wall plug source and lithium-ion batteries. The way we measured the power consumption of the system is basically by first measuring the total output current flowing through the whole systems, then using this information along with the output voltage from the battery, we can calculate the power consumption of the entire system. The calculations can be seen below:

Wall Plug Source:

Operations	Ammeter reading	Current Flowing
When valve is not opened		0.08 A
When valve is opened		1.29 A

Lithium-ion batteries:

Operations	Ammeter reading	Current Flowing
When valve is not opened		0.08 A
When valve is opened		1.15 A

Voltage generated by each source:

Power Source	Voltage
Wall-plug	12.0 V
Lithium-ion-batteries	11.1 V (3 x 3.7 V)

Power Consumption Calculation when wall-plug is used as source:

$$Total\ Power\ (valve\ closed) = 12 \times 0.08 = 0.96\ W$$

$$Total\ Power\ (valve\ opened) = 12 \times 1.29 = 15.48\ W$$

Power Consumption Calculation when lithium-ion batteries are used as source:

$$Total\ Power\ (valve\ closed) = 11.1 \times 0.08 = 0.888\ W$$

$$Total\ Power\ (valve\ opened) = 11.1 \times 1.15 = 12.765\ W$$

If we calculate the output power consumption for the lithium-ion batteries when considering the time factor using the battery current rating, we will get:

Information	Values
Current Rating	4200 mAh
Voltage per batteries	3.7 V
Total number of batteries	3

$$Total\ power\ output = 3 \times 3.7 \times 4200\ mAh = 46.64\ Wh$$

8.2 Solar Panel

When we first thought about the design, we wanted to build a system that rely mostly on solar panels instead of relying heavily on the 3-lithium ion batteries, however after further testing, we realized that, this concept wasn't really feasible since the small sized solar panel that we fitted to our system generated only a small amount of current only. Hence after further considerations, we decided that the use of the solar panel will just be to slowly charge the 3 lithium-ion batteries when they are depleted, as it cannot fully power the whole system by itself.

To run some test to find out the feasibility of using a solar panel in our system, we tested the voltage generated and also the current flowing through when an LED (light emitting diode) is used as a load under the 5:00 PM hot sun. The solar panel is faced directly at the sun and the results are measured with a digital multimeter. Below are the results and conclusions made:

Parameters measured	Results	Values
Voltage		5.85 V
Current		0.8 mA

With reference to the result, we have done a small analysis on using solar panel as power source for our system:

$$\text{Power generated by solar panel} = 5.85 \text{ V} \times 0.8 \text{ mA} = 4.68 \text{ mW}$$

However, the power needed if we use the results in the power calculation in *part 8.1* when using battery as the power source, we need approximately 12.765 W to power the opening of the valve and 0.888 W to power the system when the valve is not opening. This is not feasible as the power needed to power the whole system is significantly larger than what the solar panel can generate, as shown in the ratios below:

$$\text{Ratio of powers} = \frac{12.765}{4.68\text{m}} = 2727.56 \text{ times}$$

$$\text{Ratio of powers} = \frac{0.888}{4.68\text{m}} = 189.74 \text{ times}$$

So, the way we solve this problem to incorporate the solar panel into our system is basically to use the solar panel as a charging device to our 3 lithium-ion batteries. We also added a solar charge controller to make the whole charging process more efficient and safer. Besides that, the charge controller also allows the use of other charging devices, for examples, if the farmer wants to incorporate their own personal solar power from their own solar farm without using our build in solar panel in our product, they can do so easily.

8.3 Battery-Life

Using the information, we obtained in *part 8.1* and the characteristics of the lithium-ion batteries, we can also calculate how long the battery can last for without recharging them. The battery is used during 2 operations or mode, idle mode (where the valve is not opened) and full-on mode (where all the components in the system is running including the solenoid valve). Below we can see a full analysis on how long the battery can last without recharging them:

During Idle mode (without opening the solenoid valve), since the 3 batteries are in series, battery rating is capped at 4200 mAh:

$$\begin{aligned} \text{Battery runtime} &= \frac{\text{Battery rating}}{\text{Current flowing during idle mode}} \\ \text{Battery runtime} &= \frac{4200 \text{ mAh}}{0.08 \text{ A}} \\ \text{Battery runtime} &= 52.5 \text{ hours} \end{aligned}$$

During full-on mode (solenoid valve is opened), since the 3 batteries are in series, battery rating is capped at 4200 mAh:

$$\begin{aligned} \text{Battery runtime} &= \frac{\text{Battery rating}}{\text{Current flowing during idle mode}} \\ \text{Battery runtime} &= \frac{4200 \text{ mAh}}{1.15 \text{ A}} \\ \text{Battery runtime} &\approx 3.652 \text{ hours} \end{aligned}$$

If we estimate that the solenoid valve will be opened for **at most 30 minutes (0.5 hours) per day**:

$$\begin{aligned} \text{Idle mode per day} &= 0.08 \times 23.5 = 1.88 \text{ Ah/day} \\ \text{Valve use per day (full – on mode)} &= 1.15 \times 0.5 = 0.575 \text{ Ah/day} \\ \text{Total usage per day} &= 1.88 + 0.575 = 2.455 \text{ Ah/day} \end{aligned}$$

Since the batteries are rated at 4200 mAH, we can then find how long it will last in the unit of days:

$$\text{Full Duty Cycle} = \frac{4.2}{2.455} = 1.71 \text{ days}$$

This is equivalent to around **1 day and 17 hours** if we estimate it to be this way (solenoid valve only opened for 30 minutes per day at most) before requiring recharging.

8.4 Flow Rate and pressure test on valve

After building the product itself, we also did a flow rate test to test whether the product can efficiently handle different flow rates and pressures of water. To test this, we ran 2 tests, first by simply opening the valve and blowing air to see if air flow would be good on the other side of the pipe and secondly, we pour actual water to see if the water would flow out successfully, when the valve is turned ON.

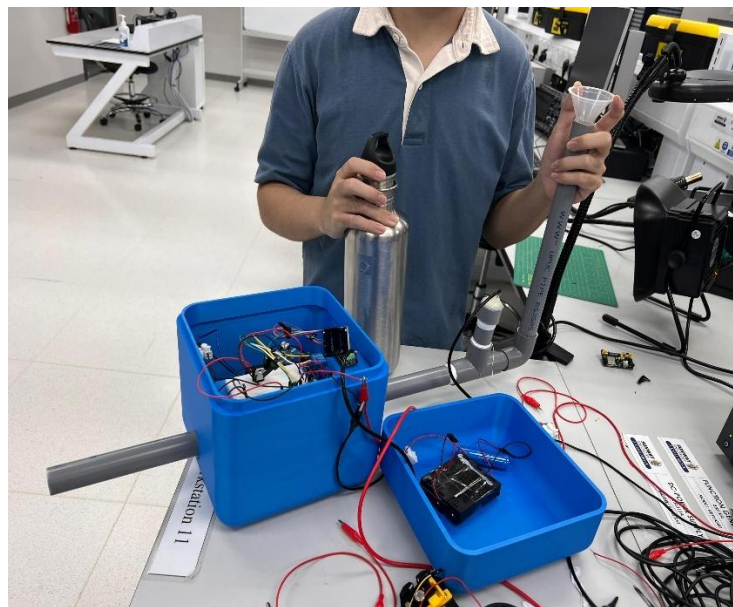


Figure 8.4.1 Flow rate test using water

After the testing of the product's flow rate, we found out that the opening and closing of the valve was good, the valve was also able to handle the water pressure if we filled it all the way up the pipe with no leakage. During the blow test, airflow was also quite good on the other side of the pipe, meaning there was no problem with the opening and closing of the valve when it came to adding resistance when water is flowing through. However there were also some downside, since we designed the pipeline to be totally horizontal as shown in the image, pressure of water must be good enough so that the water will rush out on the other side of the pipelines when the solenoid valve is opened, if the water level is low, the water pressure will be too small to actually push out the water when the solenoid valve is opened, leaving some of the water to be stuck inside the pipe lines. The way we could have solved this problem was to add a small-sized pump to help with the flow of lower pressure water. On the other hand, we could also redesign the pipelines to utilise more on gravity, instead of horizontal, we could

make it flow from high altitude to low altitude, creating difference in pressure and making the water flow more efficiently when it came to small amount of water.

8.5 Blynk IOT Software

We also tested the linking of the software we used for control and also data visualization, Blynk on both the website on a PC and using their mobile application downloaded on a phone (App Store/ Play Store).

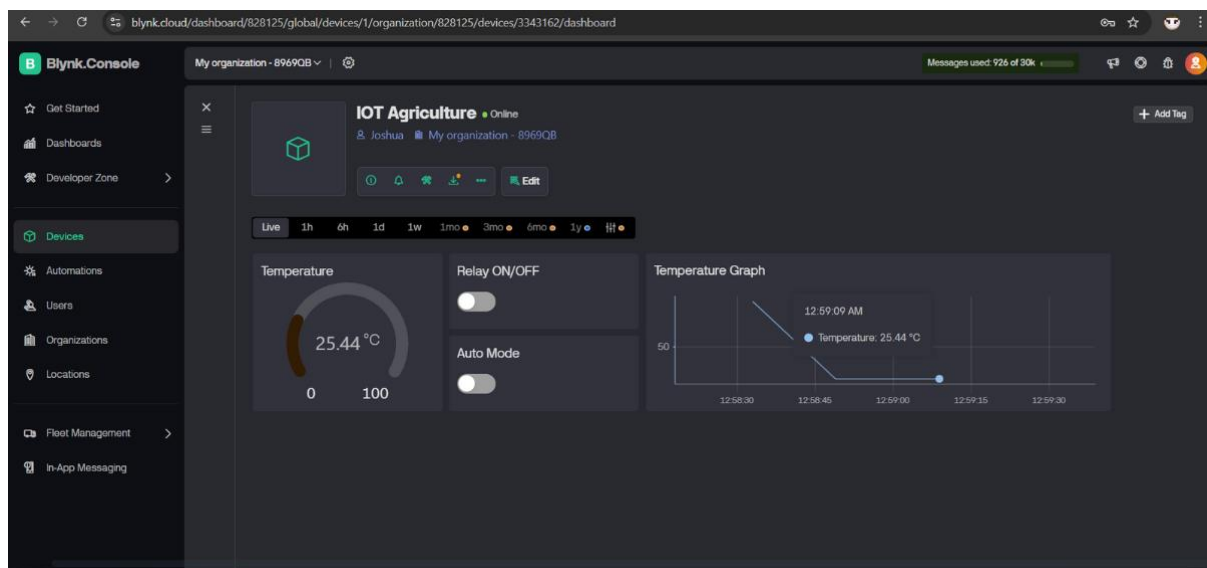


Figure 8.5.1 Blynk on Website

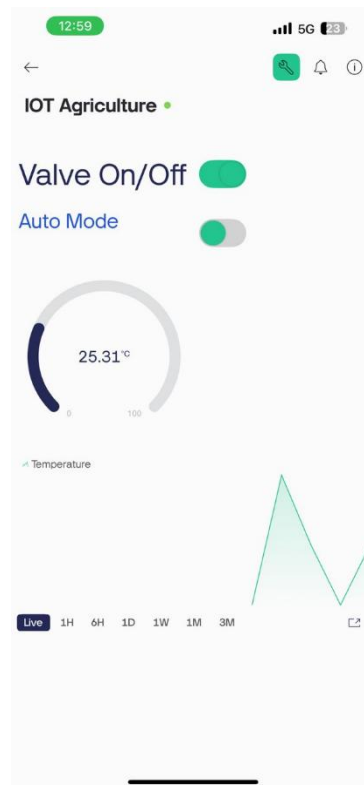


Figure 8.5.2 Blynk on Mobile Application

As seen above, everything works just like how we intended it to be, temperature values get updated to the cloud every 5 to 8 seconds and when we use GUI to manipulate the valve ON/OFF mode, it works as intended as well, same goes with the auto and manual mode, as during testing we made it so that the valve would turn ON when it reaches 30 °C and it works just fine. However, there was one problem we found out about the ESP8266 connectivity while running several tests on this linking of Blynk (IOT), is that when we put the lid of the container back on, the connectivity wasn't really the best as it would suddenly disconnect and reconnect creating lag spikes and ping delays. This lag spikes will cause the data to be updated slowly to Blynk, and the data sent by Blynk (Valve ON/OFF) would also have a latency delay as well. One improvement we can make to solve this connectivity issue is we can use an ESP8266 module with an external antenna connector (ESP-07, ESP-12F with u. FL connector). The antenna connector would be placed outside the box to better improve the connectivity of the device.

8.6 Possible Improvements

Below is the summary of the problems we encountered when doing testing and implementation on our product and what we could have done to solve some of the problems shown during the testing and implementation phase of this project:

Problems	Possible Solutions
High power consumption during full-on mode (when valve is opened)	Use a motorized ball valves which uses a small motor and require far less average power than a solenoid valve
Solar Panel's low power generation	Use a larger solar panel that can generate more power or opt for other renewable charging options such as wind turbines generation or hydroelectric
Limited Blynk Software (as to use the full version where we can have better upload latency and more cloud storage space for storing temperature data, we must pay an expensive premium subscription)	Use a hybrid approach, which means we only use Blynk for controlling the valve and for the limitation of data storage for temperature, we can solve this by using Thingspeak, so basically by integrating both software at the same time or we can build our own web dashboard and app that is specifically tailored to our product
Weak Wi-Fi Connectivity	Use an external antenna connector that would improve Wi-Fi connectivity or rely less on Wi-Fi and instead opt for other alternatives, such as LoRa (Long Range radio), wired connection or Bluetooth
Water stuck if low water level flowing through pipelines	Integrate a pump into the system or utilize gravity by redesigning the pipeline to flow from top to bottom instead of horizontal
How can we make the system smarter in a way that would improve efficiency and lower power consumption	Integrate Artificial Intelligence system (AI) into the whole system, for example small modes such as decision tree, tiny neural network or other small machine learning models that has a build in smart scheduling system (basically a system that can automatically minimize valve hold time while maintaining irrigation needs, avoid unnecessary uploads that can save Blynk quota and power and exploit sunny periods to schedule high-power usage events such as opening the valve)

9.0 Closing Remarks

In conclusion, this final report written by us marks the end and the successful completion of our project on the automatic irrigation farm water control system, which has been accomplished smoothly and efficiently in terms of scope, time and cost. Through the application of project management techniques, methods and specific engineering design skills we learned in our engineering course, all our objectives were achieved as planned. The excellent coordination and immense commitment of all team members throughout every phase of CDIO (conceive, design, implement and operate) played a crucial role in the successful design, development and implementation of this product.

The main outlines, objectives and overall deliverables were fully realized, while potential risks were addressed as well during the operation and testing phases. The final system not only successfully meets the requirements set by our stakeholders but beyond that, it also exceeded our initial expectations. Even though there were many challenges, but we addressed them with passion one by one.

With this achievement, this has established a strong foundation for future improvements to address some of the problems during the testing phase to make the product even better and potential scalability. Overall, we are also confident that the outcome of our creation will provide real value for practical applications in the agriculture sector.